

MANUEL PROFESSIONNEL ET UNIVERSITAIRE

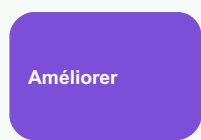
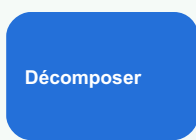
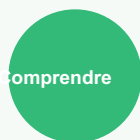
Consultant en innovation technologique, transformation numérique, cybersécurité, intelligence artificielle, développement logiciel et stratégie digitale.

ALGORITHMIQUE

Contact	Coordonnées
Site web	www.charmantnyungu.com
Email	consultant@charmantnyungu.com
Téléphone	+243 835 137 837
Édition	Formation complète — 2026

Charmant Nyungu

Vision du cours



Ce manuel apprend à réfléchir comme un développeur : analyser, structurer, prouver, coder et améliorer.

Presentation du manuel

Ce manuel d'algorithmique et logique de programmation apprend à réfléchir comme un développeur. Il accompagne l'apprenant dans les conditions, boucles, tableaux, fonctions, tris, recherches, complexité et méthodes de résolution de problèmes.

L'auteur, Charmant Nyungu, propose une formation progressive pour les débutants, étudiants, développeurs juniors, professionnels en reconversion et toute personne qui veut comprendre la logique avant de dépendre d'un framework ou d'un langage particulier.

Élément	Détail
Auteur	Charmant Nyungu
Profession	Consultant en innovation technologique, transformation numérique, cybersécurité, intelligence artificielle, développement logiciel et
Site	www.charmantnyungu.com
Email	consultant@charmantnyungu.com
Téléphone	+243 835 137 837

Préface

L'algorithmique est la grammaire profonde de la programmation. Un développeur fort n'est pas celui qui connaît seulement une syntaxe, mais celui qui sait découper un problème, choisir une stratégie, mesurer le coût et expliquer pourquoi sa solution fonctionne.

Comment utiliser ce livre

- Lire chaque problème avant de regarder le code.
- Écrire le pseudo-code à la main avant Python.
- Tester les cas simples, limites et impossibles.
- Comparer plusieurs solutions selon leur complexité.
- Refaire les exercices jusqu'à pouvoir expliquer la logique oralement.

Table des matières détaillée

1. Partie 1 : Penser comme un développeur

- Logique de résolution : problème, entrée, traitement, sortie
- Pseudo-code et organigrammes : instruction, condition, boucle, étape
- Décomposition : sous-problème, fonction, responsabilité, test

2. Partie 2 : Instructions de base

- Variables et expressions : valeur, type, affectation, calcul
- Conditions : if, else, comparaison, branche
- Boucles : for, while, compteur, arrêt
- Fonctions : paramètre, retour, contrat, réutilisation

3. Partie 3 : Structures de données

- Tableaux et listes : indice, parcours, insertion, suppression
- Chaînes de caractères : caractère, parcours, recherche, transformation
- Piles et files : LIFO, FIFO, push, pop
- Dictionnaires et ensembles : clé, valeur, hash, unicité

4. Partie 4 : Recherche et tri

- Recherche linéaire : parcours, comparaison, position, absence
- Recherche binaire : tri préalable, milieu, division, logarithme
- Tri à bulles et sélection : comparaison, échange, minimum, coût
- Tri insertion et fusion : insertion, division, fusion, stabilité
- Choisir un algorithme de tri : taille, mémoire, stabilité, performance

5. Partie 5 : Complexité algorithmique

- Notion de coût : temps, mémoire, taille, opération
- Notation Big O : $O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$
- Optimisation raisonnée : mesure, profiling, compromis, lisibilité

6. Partie 6 : Techniques de résolution

- Deux pointeurs : gauche, droite, fenêtre, condition
- Fenêtre glissante : segment, somme, maximum, mise à jour
- Récursivité : cas de base, appel, pile, retour
- Programmation dynamique : sous-problème, mémorisation, table, optimalité
- Graphes introductifs : sommet, arête, parcours, chemin

7. Partie 7 : Projets pratiques

- Moteur de recherche simple : texte, index, score, résultat
- Analyseur de transactions : liste, filtre, somme, alerte
- Planificateur de tâches : priorité, file, temps, ordre
- Mini système de recommandation : similarité, score, tri, top
- Jeu de logique : état, règle, boucle, victoire

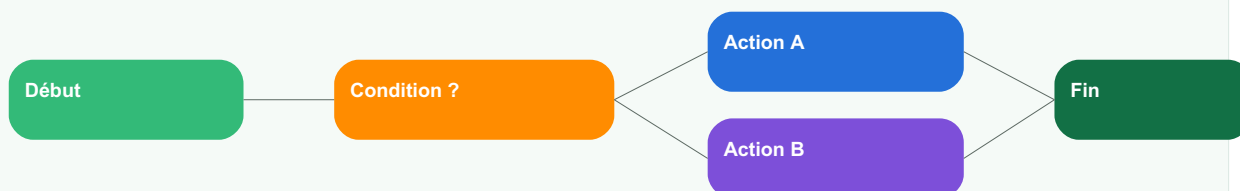
8. Partie 8 : Exercices, examens et annexes

- Banque d'exercices : conditions, boucles, tableaux, complexité
- Examens blancs : problèmes, preuve, code, analyse
- Glossaire et feuille de route : termes, méthode, progression, pratique

Logique de résolution

Ce chapitre traite de logique de résolution pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Décision algorithmique



Objectifs pédagogiques

- Comprendre le rôle de logique de résolution dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
problème	Comment utiliser problème pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
entrée	Comment utiliser entrée pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
traitement	Comment utiliser traitement pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
sortie	Comment utiliser sortie pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

problème

problème est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. problème correspond à l'un de ces éléments de décision.

Tableau et indices**entrée**

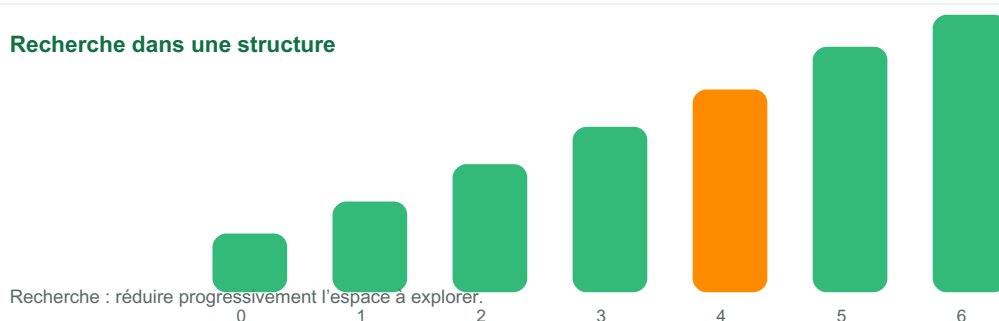
entrée est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. entrée correspond à l'un de ces éléments de décision.

traitement

traitement est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. traitement correspond à l'un de ces éléments de décision.

Recherche dans une structure**sortie**

sortie est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. sortie correspond à l'un de ces éléments de décision.

Exemple de code

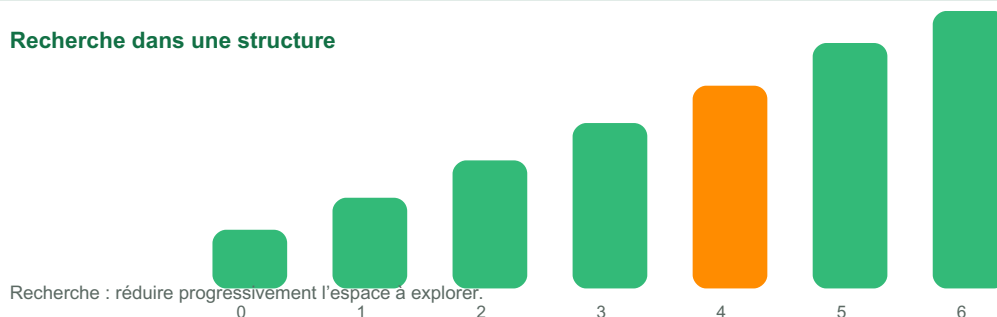
Code — Logique de résolution

```
def resoudre(donnees):
    resultat = []
    for element in donnees:
        if element is not None:
            resultat.append(element)
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Recherche dans une structure

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à problème. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à entrée. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à traitement. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à sortie. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à problème. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à entrée. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Pseudo-code et organigrammes

Ce chapitre traite de pseudo-code et organigrammes pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Tableau et indices



Objectifs pédagogiques

- Comprendre le rôle de pseudo-code et organigrammes dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
instruction	Comment utiliser instruction pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
condition	Comment utiliser condition pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
boucle	Comment utiliser boucle pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
étape	Comment utiliser étape pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

instruction

instruction est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. instruction correspond à l'un de ces éléments de décision.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

condition

condition est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

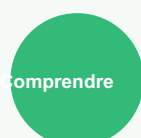
Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. condition correspond à l'un de ces éléments de décision.

boucle

boucle est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. boucle correspond à l'un de ces éléments de décision.

Méthode de résolution



Décomposer

Tester

Améliorer

étape

étape est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. étape correspond à l'un de ces éléments de décision.

Exemple de code

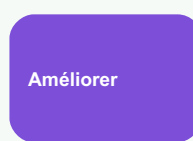
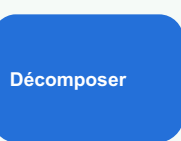
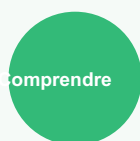
Code — Pseudo-code et organigrammes

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Méthode de résolution

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à instruction. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à condition. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à boucle. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à étape. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à instruction. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à condition. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Decomposition

Ce chapitre traite de décomposition pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Objectifs pédagogiques

- Comprendre le rôle de décomposition dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
sous-problème	Comment utiliser sous-problème pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
fonction	Comment utiliser fonction pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
responsabilité	Comment utiliser responsabilité pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
test	Comment utiliser test pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

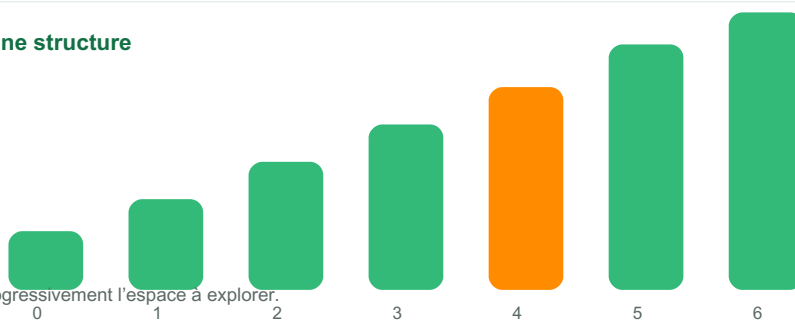
sous-problème

sous-problème est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. sous-problème correspond à l'un de ces éléments de décision.

Recherche dans une structure

Recherche : réduire progressivement l'espace à explorer.

**fonction**

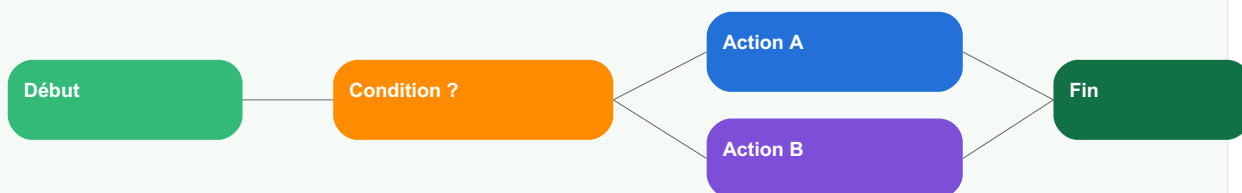
fonction est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. fonction correspond à l'un de ces éléments de décision.

responsabilité

responsabilité est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. responsabilité correspond à l'un de ces éléments de décision.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

test

test est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. test correspond à l'un de ces éléments de décision.

Exemple de code

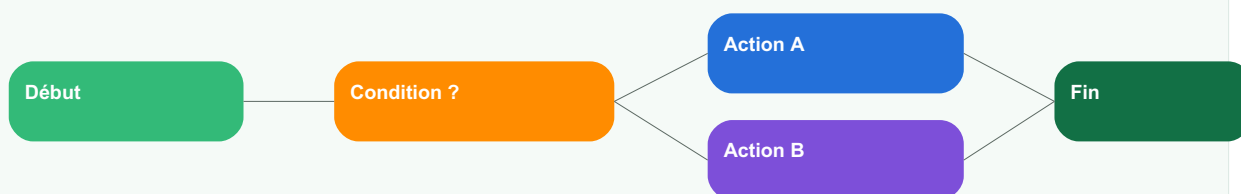
Code — Décomposition

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à sous-problème. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à fonction. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à responsabilité. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à test. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à sous-problème. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à fonction. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

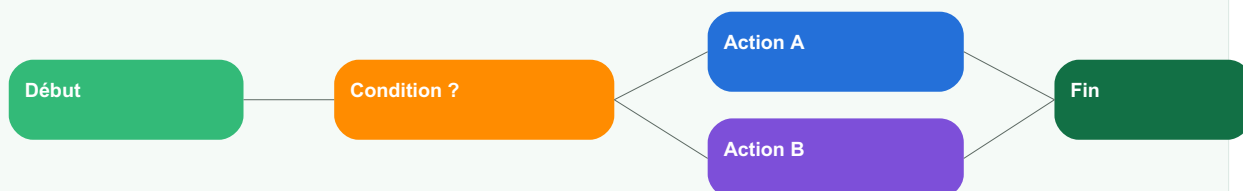
Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

variables et expressions

Ce chapitre traite de variables et expressions pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Décision algorithmique



Objectifs pédagogiques

- Comprendre le rôle de variables et expressions dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
valeur	Comment utiliser valeur pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
type	Comment utiliser type pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
affectation	Comment utiliser affectation pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
calcul	Comment utiliser calcul pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

valeur

valeur est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. valeur correspond à l'un de ces éléments de décision.

Tableau et indices**type**

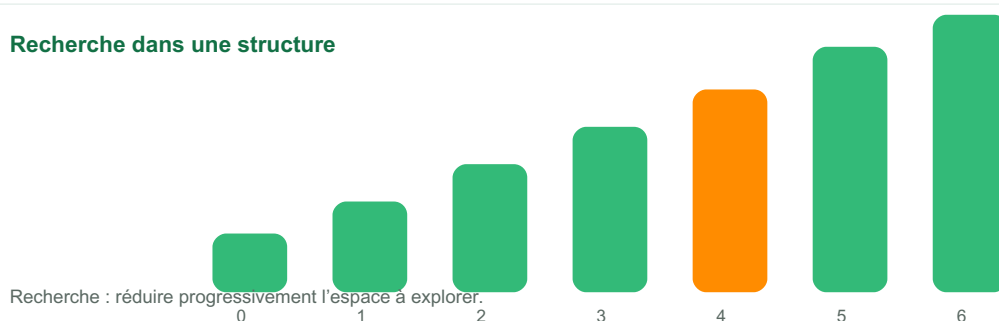
type est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. type correspond à l'un de ces éléments de décision.

affectation

affectation est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. affectation correspond à l'un de ces éléments de décision.

Recherche dans une structure**calcul**

calcul est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. calcul correspond à l'un de ces éléments de décision.

Exemple de code

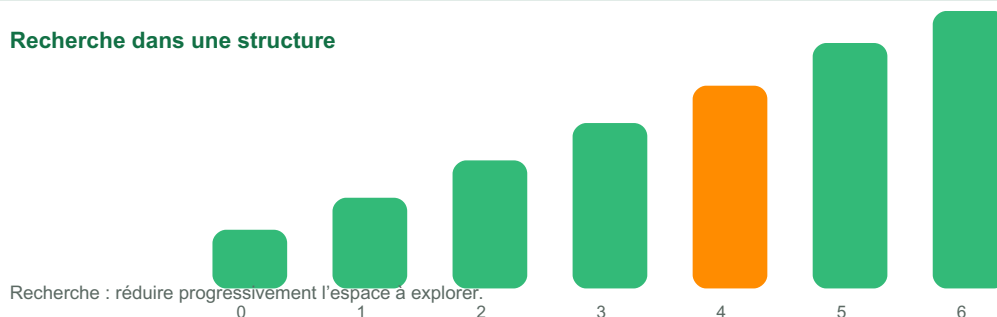
Code — Variables et expressions

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Recherche dans une structure

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à valeur. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à type. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à affectation. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à calcul. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à valeur. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à type. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Conditions

Ce chapitre traite de conditions pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Tableau et indices



Objectifs pédagogiques

- Comprendre le rôle de conditions dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
if	Comment utiliser if pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
else	Comment utiliser else pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
comparaison	Comment utiliser comparaison pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
branche	Comment utiliser branche pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

if

if est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. if correspond à l'un de ces éléments de décision.

Courbes de complexité

Complexité : comparer le coût quand la taille des données augmente.

else

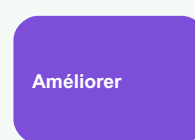
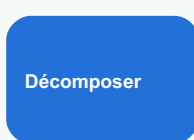
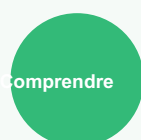
else est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. else correspond à l'un de ces éléments de décision.

comparaison

comparaison est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. comparaison correspond à l'un de ces éléments de décision.

Méthode de résolution**branche**

branche est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. branche correspond à l'un de ces éléments de décision.

Exemple de code

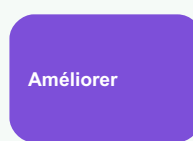
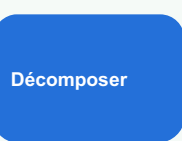
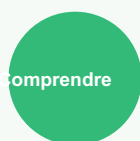
Code — Conditions

```
def statut_note(note):  
    if note >= 80:  
        return "excellent"  
    if note >= 50:  
        return "admis"  
    return "à reprendre"
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Méthode de résolution

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à if. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à else. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à comparaison. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à branche. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à if. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à else. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Boucles

Ce chapitre traite de boucles pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Objectifs pédagogiques

- Comprendre le rôle de boucles dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
for	Comment utiliser for pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
while	Comment utiliser while pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
compteur	Comment utiliser compteur pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
arrêt	Comment utiliser arrêt pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

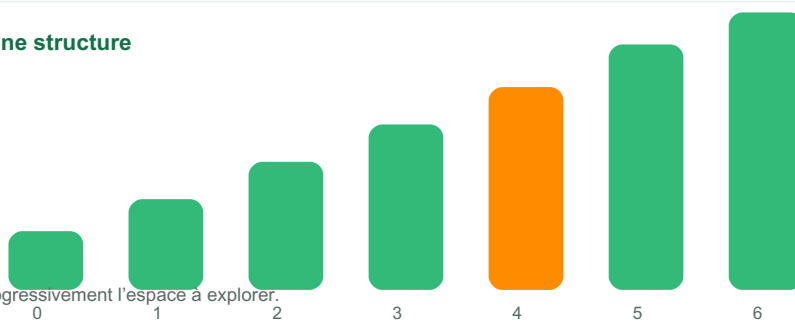
for

for est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. for correspond à l'un de ces éléments de décision.

Recherche dans une structure

Recherche : réduire progressivement l'espace à explorer.

**while**

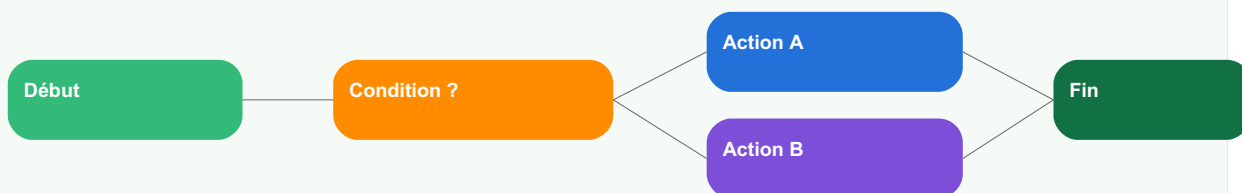
while est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. while correspond à l'un de ces éléments de décision.

compteur

compteur est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. compteur correspond à l'un de ces éléments de décision.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

arrêt

arrêt est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. arrêt correspond à l'un de ces éléments de décision.

Exemple de code

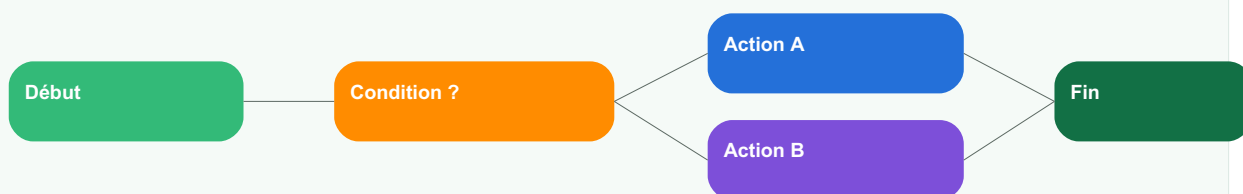
Code — Boucles

```
def somme_paires(n):  
    total = 0  
    for nombre in range(1, n + 1):  
        if nombre % 2 == 0:  
            total += nombre  
    return total
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à for. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à while. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à compteur. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à arrêt. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à for. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à while. Donnez le pseudo-code, le code Python, trois tests et la complexité.

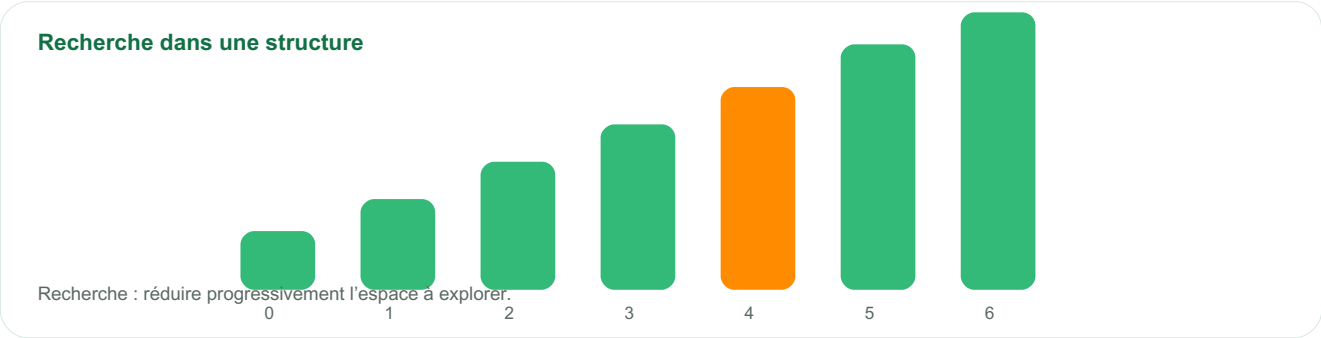
Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Fonctions

Ce chapitre traite de fonctions pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.



Objectifs pédagogiques

- Comprendre le rôle de fonctions dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

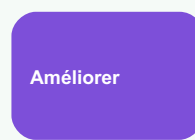
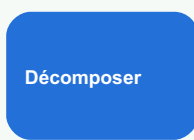
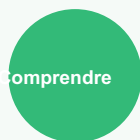
Notion	Question logique	Résultat attendu
paramètre	Comment utiliser paramètre pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
retour	Comment utiliser retour pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
contrat	Comment utiliser contrat pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
réutilisation	Comment utiliser réutilisation pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

paramètre

paramètre est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. paramètre correspond à l'un de ces éléments de décision.

Méthode de résolution**retour**

retour est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. retour correspond à l'un de ces éléments de décision.

contrat

contrat est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. contrat correspond à l'un de ces éléments de décision.

Tableau et indices

12	5	21	9	1	17	30	3
0	1	2	3	4	5	6	7

réutilisation

réutilisation est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. réutilisation correspond à l'un de ces éléments de décision.

Exemple de code

Code — Fonctions

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Tableau et indices

12	5	21	9	1	17	30	3
0	1	2	3	4	5	6	7

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à paramètre. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à retour. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à contrat. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à réutilisation. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à paramètre. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à retour. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

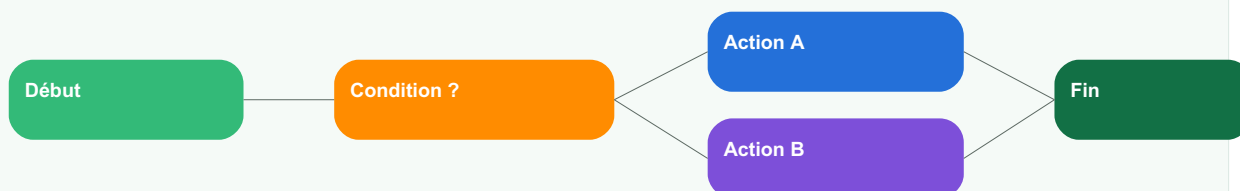
Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Tableaux et listes

Ce chapitre traite de tableaux et listes pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Décision algorithmique



Organigramme : visualiser décision, branches et sortie.

Objectifs pédagogiques

- Comprendre le rôle de tableaux et listes dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
indice	Comment utiliser indice pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
parcours	Comment utiliser parcours pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
insertion	Comment utiliser insertion pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
suppression	Comment utiliser suppression pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

indice

indice est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. indice correspond à l'un de ces éléments de décision.

Tableau et indices**parcours**

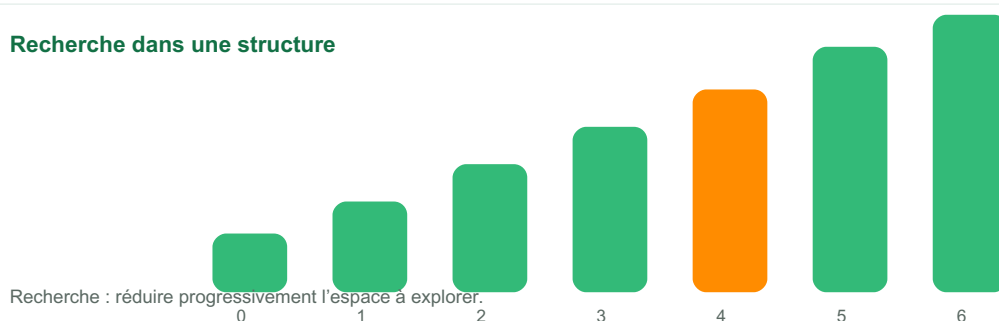
parcours est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. parcours correspond à l'un de ces éléments de décision.

insertion

insertion est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. insertion correspond à l'un de ces éléments de décision.

Recherche dans une structure**suppression**

suppression est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. suppression correspond à l'un de ces éléments de décision.

Exemple de code

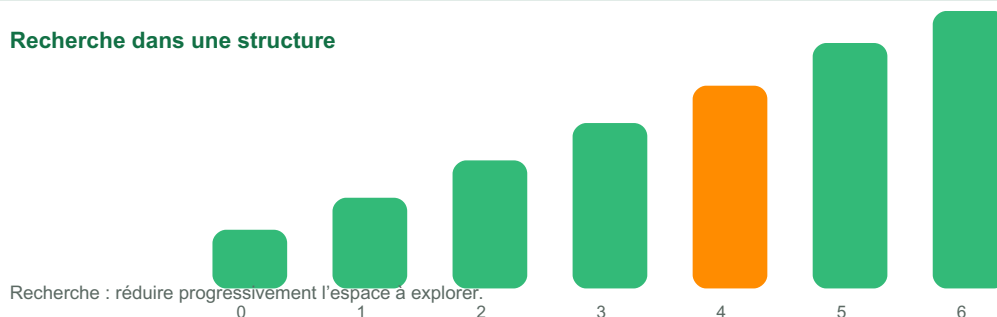
Code — Tableaux et listes

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Recherche dans une structure

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à indice. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à parcours. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à insertion. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à suppression. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à indice. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à parcours. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Chaines de caracteres

Ce chapitre traite de chaînes de caractères pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Tableau et indices



Objectifs pédagogiques

- Comprendre le rôle de chaînes de caractères dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
caractère	Comment utiliser caractère pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
parcours	Comment utiliser parcours pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
recherche	Comment utiliser recherche pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
transformation	Comment utiliser transformation pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

caractère

caractère est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. caractère correspond à l'un de ces éléments de décision.

Courbes de complexité

Complexité : comparer le coût quand la taille des données augmente.

parcours

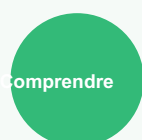
parcours est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. parcours correspond à l'un de ces éléments de décision.

recherche

recherche est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. recherche correspond à l'un de ces éléments de décision.

Méthode de résolution

Décomposer

Tester

Améliorer

transformation

transformation est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. transformation correspond à l'un de ces éléments de décision.

Exemple de code

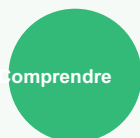
Code — Chaînes de caractères

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Méthode de résolution

Décomposer



Améliorer

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à caractère. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à parcours. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à recherche. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à transformation. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à caractère. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à parcours. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

PILES ET TILES

Ce chapitre traite de piles et files pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Objectifs pédagogiques

- Comprendre le rôle de piles et files dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
LIFO	Comment utiliser LIFO pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
FIFO	Comment utiliser FIFO pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
push	Comment utiliser push pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
pop	Comment utiliser pop pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

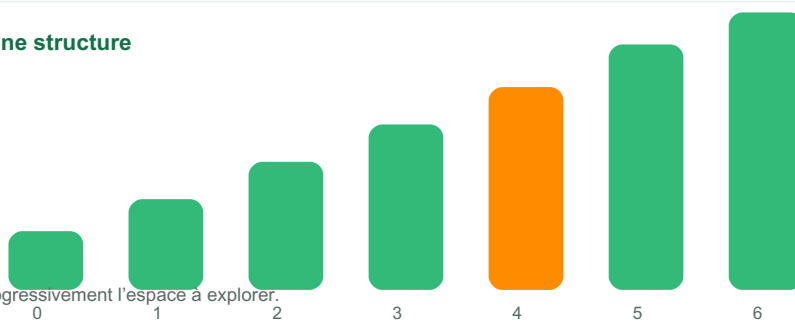
LIFO

LIFO est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. LIFO correspond à l'un de ces éléments de décision.

Recherche dans une structure

Recherche : réduire progressivement l'espace à explorer.

**FIFO**

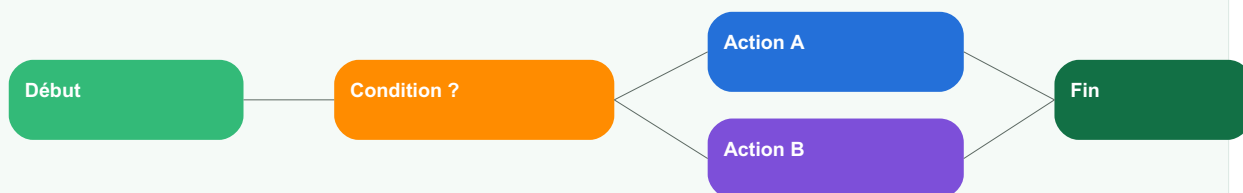
FIFO est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. FIFO correspond à l'un de ces éléments de décision.

push

push est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. push correspond à l'un de ces éléments de décision.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

pop

pop est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. pop correspond à l'un de ces éléments de décision.

Exemple de code

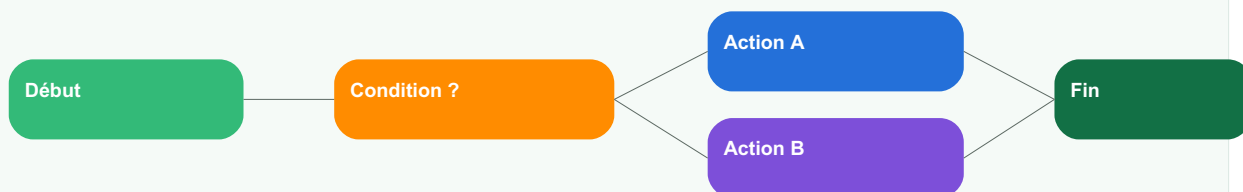
Code — Piles et files

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à LIFO. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à FIFO. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à push. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à pop. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à LIFO. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à FIFO. Donnez le pseudo-code, le code Python, trois tests et la complexité.

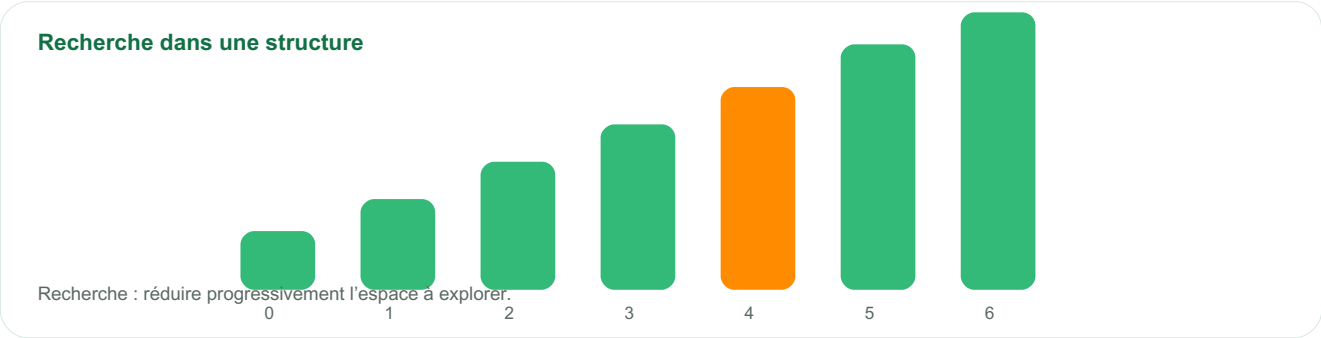
Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Dictionnaires et ensembles

Ce chapitre traite de dictionnaires et ensembles pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.



Objectifs pédagogiques

- Comprendre le rôle de dictionnaires et ensembles dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

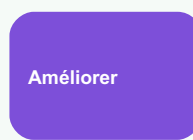
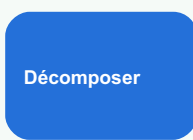
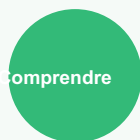
Notion	Question logique	Résultat attendu
clé	Comment utiliser clé pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
valeur	Comment utiliser valeur pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
hash	Comment utiliser hash pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
unicité	Comment utiliser unicité pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

clé

clé est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. clé correspond à l'un de ces éléments de décision.

Méthode de résolution**valeur**

valeur est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. valeur correspond à l'un de ces éléments de décision.

hash

hash est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. hash correspond à l'un de ces éléments de décision.

Tableau et indices**unicité**

unicité est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. unicité correspond à l'un de ces éléments de décision.

Exemple de code

Code — Dictionnaires et ensembles

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Tableau et indices

12	5	21	9	1	17	30	3
0	1	2	3	4	5	6	7

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à clé. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à valeur. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à hash. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à unicité. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à clé. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à valeur. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

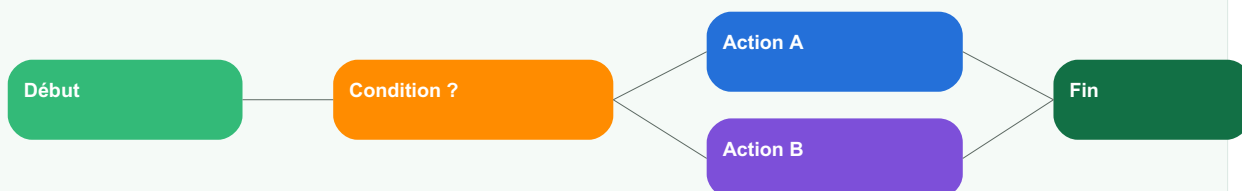
Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Recherche linéaire

Ce chapitre traite de recherche linéaire pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Décision algorithmique



Objectifs pédagogiques

- Comprendre le rôle de recherche linéaire dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
parcours	Comment utiliser parcours pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
comparaison	Comment utiliser comparaison pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
position	Comment utiliser position pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
absence	Comment utiliser absence pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

parcours

parcours est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. parcours correspond à l'un de ces éléments de décision.

Tableau et indices**comparaison**

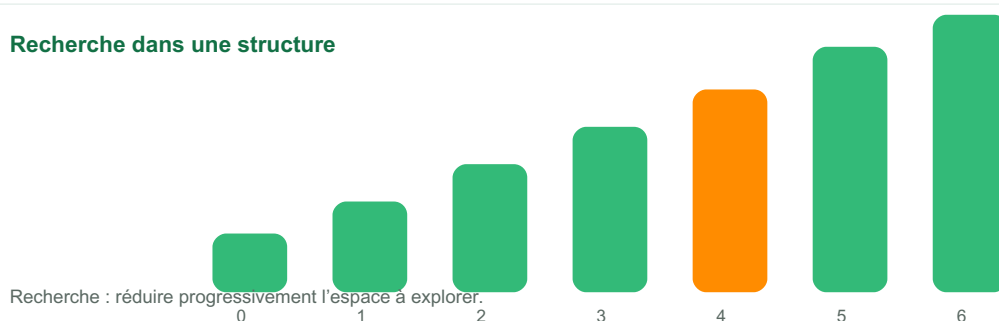
comparaison est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. comparaison correspond à l'un de ces éléments de décision.

position

position est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. position correspond à l'un de ces éléments de décision.

Recherche dans une structure**absence**

absence est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. absence correspond à l'un de ces éléments de décision.

Exemple de code

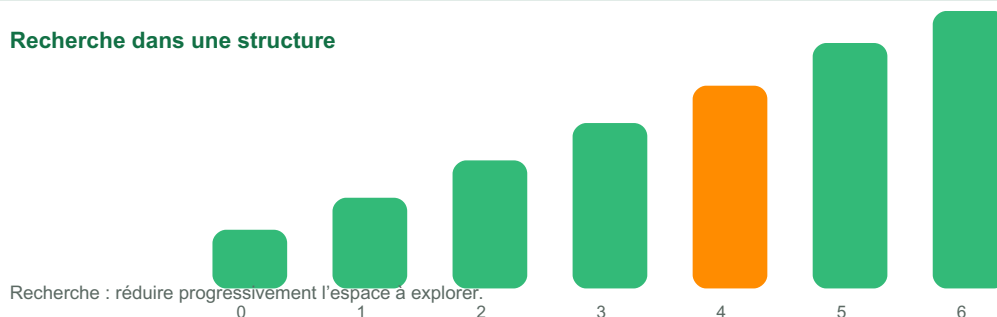
Code — Recherche linéaire

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Recherche dans une structure

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à parcours. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à comparaison. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à position. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à absence. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à parcours. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à comparaison. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Recherche binaire

Ce chapitre traite de recherche binaire pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Tableau et indices



Objectifs pédagogiques

- Comprendre le rôle de recherche binaire dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
tri préalable	Comment utiliser tri préalable pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
milieu	Comment utiliser milieu pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
division	Comment utiliser division pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
logarithme	Comment utiliser logarithme pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

tri préalable

tri préalable est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. tri préalable correspond à l'un de ces éléments de décision.

Courbes de complexité

Complexité : comparer le coût quand la taille des données augmente.

milieu

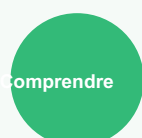
milieu est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. milieu correspond à l'un de ces éléments de décision.

division

division est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. division correspond à l'un de ces éléments de décision.

Méthode de résolution

Décomposer

Tester

Améliorer

logarithme

logarithme est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. logarithme correspond à l'un de ces éléments de décision.

Exemple de code

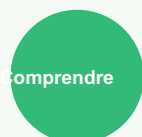
Code — Recherche binaire

```
def recherche_binaire(tableau, cible):  
    gauche, droite = 0, len(tableau) - 1  
    while gauche <= droite:  
        milieu = (gauche + droite) // 2  
        if tableau[milieu] == cible:  
            return milieu  
        if tableau[milieu] < cible:  
            gauche = milieu + 1  
        else:  
            droite = milieu - 1  
    return -1
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Méthode de résolution**Décomposer****Tester****Améliorer**

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à tri préalable. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à milieu. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à division. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à logarithme. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à tri préalable. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à milieu. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Tri à bulles et sélection

Ce chapitre traite de tri à bulles et sélection pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Objectifs pédagogiques

- Comprendre le rôle de tri à bulles et sélection dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
comparaison	Comment utiliser comparaison pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
échange	Comment utiliser échange pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
minimum	Comment utiliser minimum pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
coût	Comment utiliser coût pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

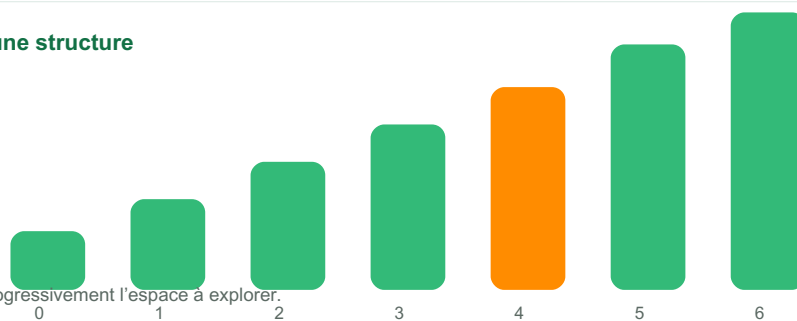
comparaison

comparaison est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. comparaison correspond à l'un de ces éléments de décision.

Recherche dans une structure

Recherche : réduire progressivement l'espace à explorer.

**échange**

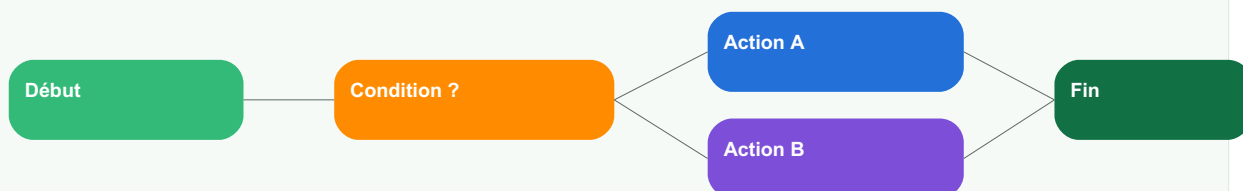
échange est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. échange correspond à l'un de ces éléments de décision.

minimum

minimum est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. minimum correspond à l'un de ces éléments de décision.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

coût

coût est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. coût correspond à l'un de ces éléments de décision.

Exemple de code

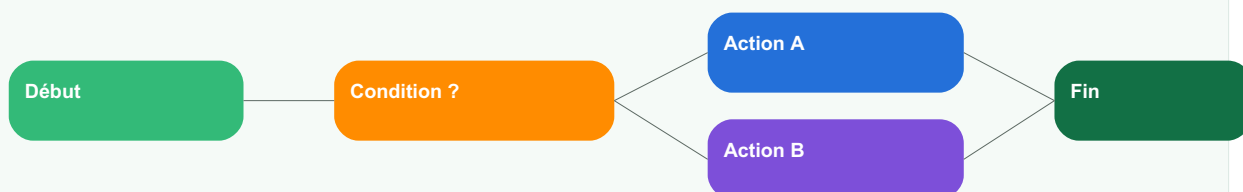
Code — Tri à bulles et sélection

```
def tri_insertion(valeurs):  
    for i in range(1, len(valeurs)):  
        cle = valeurs[i]  
        j = i - 1  
        while j >= 0 and valeurs[j] > cle:  
            valeurs[j + 1] = valeurs[j]  
            j -= 1  
        valeurs[j + 1] = cle  
    return valeurs
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à comparaison. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à échange. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à minimum. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à coût. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à comparaison. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à échange. Donnez le pseudo-code, le code Python, trois tests et la complexité.

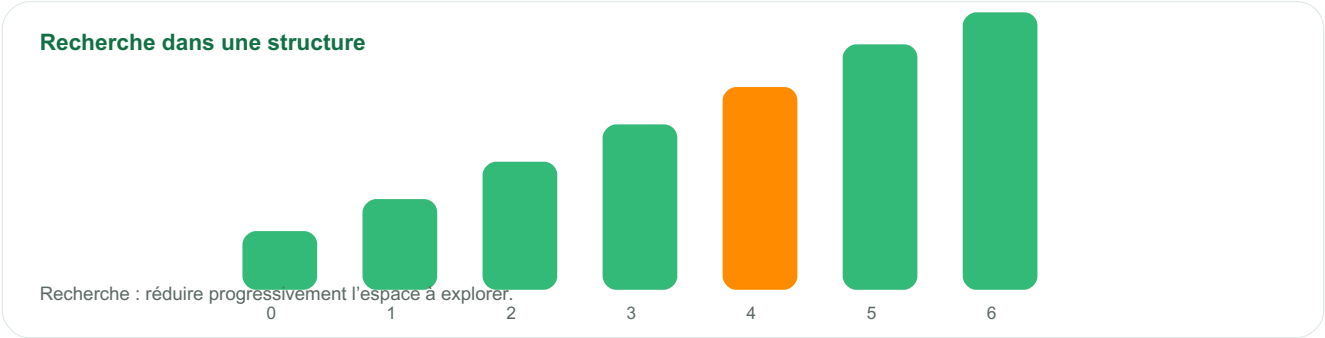
Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

TRI INSERTION ET FUSION

Ce chapitre traite de tri insertion et fusion pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.



Objectifs pédagogiques

- Comprendre le rôle de tri insertion et fusion dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

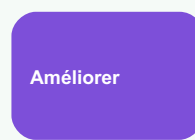
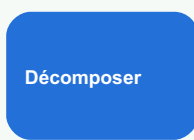
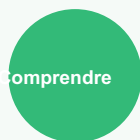
Notion	Question logique	Résultat attendu
insertion	Comment utiliser insertion pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
division	Comment utiliser division pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
fusion	Comment utiliser fusion pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
stabilité	Comment utiliser stabilité pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

insertion

insertion est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. insertion correspond à l'un de ces éléments de décision.

Méthode de résolution**division**

division est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. division correspond à l'un de ces éléments de décision.

fusion

fusion est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. fusion correspond à l'un de ces éléments de décision.

Tableau et indices

12	5	21	9	1	17	30	3
0	1	2	3	4	5	6	7

stabilité

stabilité est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. stabilité correspond à l'un de ces éléments de décision.

Exemple de code

Code — Tri insertion et fusion

```
def tri_insertion(valeurs):  
    for i in range(1, len(valeurs)):  
        cle = valeurs[i]  
        j = i - 1  
        while j >= 0 and valeurs[j] > cle:  
            valeurs[j + 1] = valeurs[j]  
            j -= 1  
        valeurs[j + 1] = cle  
    return valeurs
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Tableau et indices

12	5	21	9	1	17	30	3
0	1	2	3	4	5	6	7

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à insertion. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à division. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à fusion. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à stabilité. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à insertion. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à division. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

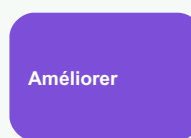
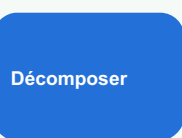
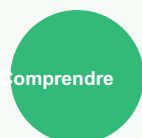
Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Choisir un algorithme de tri

Ce chapitre traite de choisir un algorithme de tri pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Méthode de résolution



Objectifs pédagogiques

- Comprendre le rôle de choisir un algorithme de tri dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

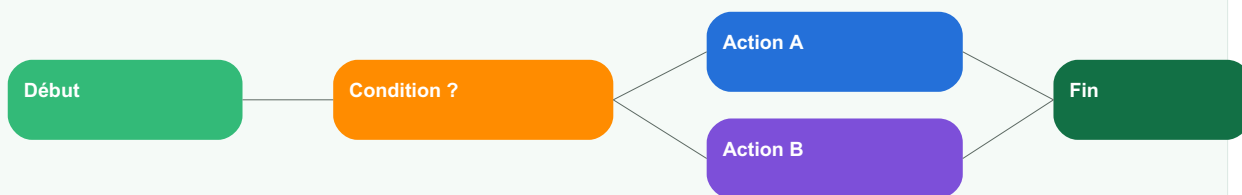
Notion	Question logique	Résultat attendu
taille	Comment utiliser taille pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
mémoire	Comment utiliser mémoire pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
stabilité	Comment utiliser stabilité pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
performance	Comment utiliser performance pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

taille

taille est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. taille correspond à l'un de ces éléments de décision.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

mémoire

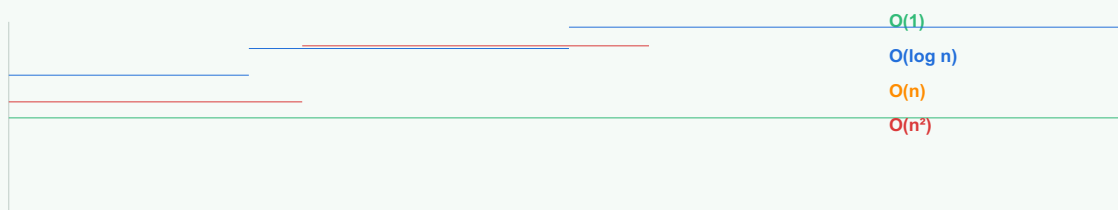
mémoire est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. mémoire correspond à l'un de ces éléments de décision.

stabilité

stabilité est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. stabilité correspond à l'un de ces éléments de décision.

Courbes de complexité

Complexité : comparer le coût quand la taille des données augmente.

performance

performance est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. performance correspond à l'un de ces éléments de décision.

Exemple de code

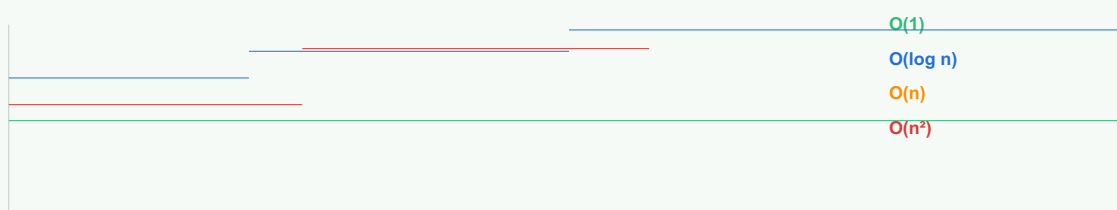
Code — Choisir un algorithme de tri

```
def tri_insertion(valeurs):
    for i in range(1, len(valeurs)):
        cle = valeurs[i]
        j = i - 1
        while j >= 0 and valeurs[j] > cle:
            valeurs[j + 1] = valeurs[j]
            j -= 1
        valeurs[j + 1] = cle
    return valeurs
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Courbes de complexité

Complexité : comparer le coût quand la taille des données augmente.

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à taille. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à mémoire. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à stabilité. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à performance. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à taille. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à mémoire. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

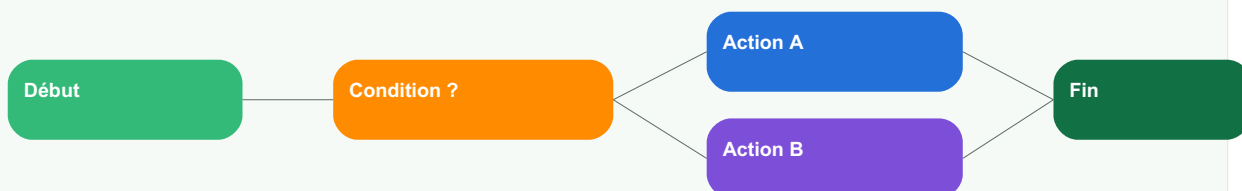
Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

NOTION DE COUT

Ce chapitre traite de notion de coût pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Décision algorithmique



Objectifs pédagogiques

- Comprendre le rôle de notion de coût dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
temps	Comment utiliser temps pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
mémoire	Comment utiliser mémoire pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
taille	Comment utiliser taille pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
opération	Comment utiliser opération pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

temps

temps est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. temps correspond à l'un de ces éléments de décision.

Tableau et indices**mémoire**

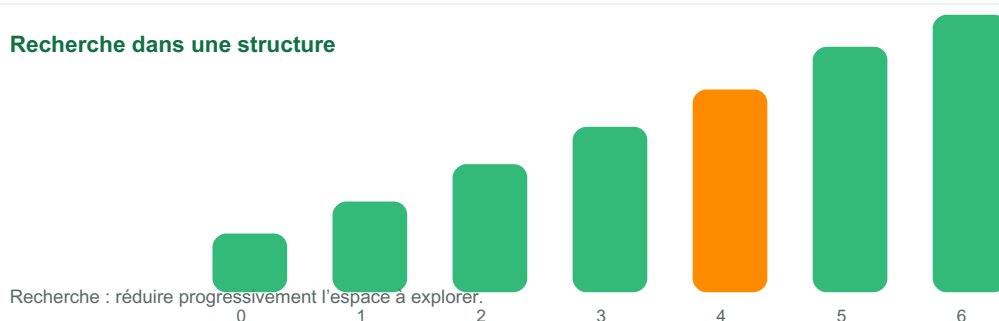
mémoire est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. mémoire correspond à l'un de ces éléments de décision.

taille

taille est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. taille correspond à l'un de ces éléments de décision.

Recherche dans une structure**opération**

opération est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. opération correspond à l'un de ces éléments de décision.

Exemple de code

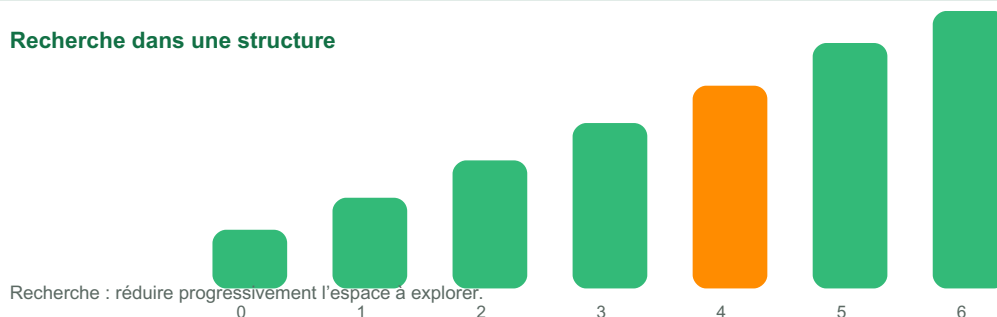
Code — Notion de coût

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Recherche dans une structure

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à temps. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à mémoire. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à taille. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à opération. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à temps. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à mémoire. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Notation Big O

Ce chapitre traite de notation big o pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Tableau et indices

12	5	21	9	1	17	30	3
0	1	2	3	4	5	6	7

Objectifs pédagogiques

- Comprendre le rôle de notation big o dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
$O(1)$	Comment utiliser $O(1)$ pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
$O(\log n)$	Comment utiliser $O(\log n)$ pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
$O(n)$	Comment utiliser $O(n)$ pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
$O(n^2)$	Comment utiliser $O(n^2)$ pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

$O(1)$

$O(1)$ est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. $O(1)$ correspond à l'un de ces éléments de décision.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

$O(\log n)$

$O(\log n)$ est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

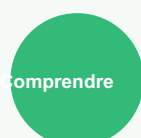
Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. $O(\log n)$ correspond à l'un de ces éléments de décision.

$O(n)$

$O(n)$ est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. $O(n)$ correspond à l'un de ces éléments de décision.

Méthode de résolution



Décomposer

Tester

Améliorer

$O(n^2)$

$O(n^2)$ est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. $O(n^2)$ correspond à l'un de ces éléments de décision.

Exemple de code

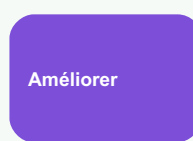
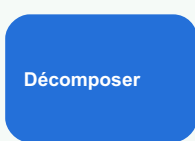
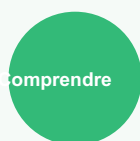
Code — Notation Big O

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Méthode de résolution

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à $O(1)$. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à $O(\log n)$. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à $O(n)$. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à $O(n^2)$. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à $O(1)$. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à $O(\log n)$. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Optimisation raisonnée

Ce chapitre traite de optimisation raisonnée pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Objectifs pédagogiques

- Comprendre le rôle de optimisation raisonnée dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
mesure	Comment utiliser mesure pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
profiling	Comment utiliser profiling pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
compromis	Comment utiliser compromis pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
lisibilité	Comment utiliser lisibilité pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

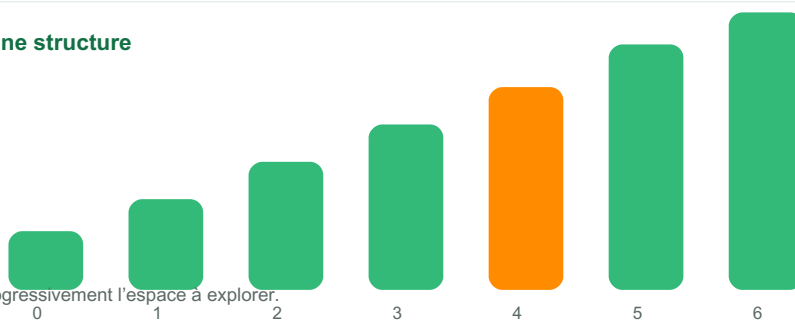
mesure

mesure est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. mesure correspond à l'un de ces éléments de décision.

Recherche dans une structure

Recherche : réduire progressivement l'espace à explorer.

**profiling**

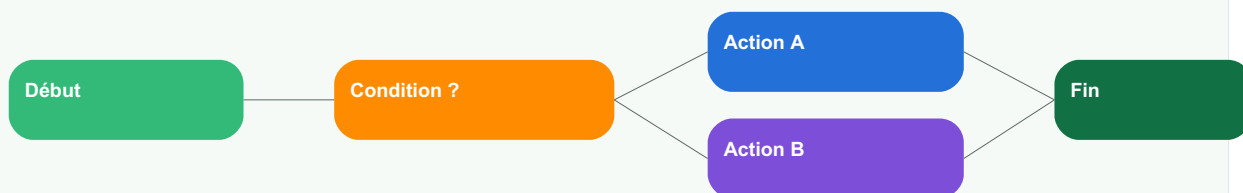
profiling est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. profiling correspond à l'un de ces éléments de décision.

compromis

compromis est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. compromis correspond à l'un de ces éléments de décision.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

lisibilité

lisibilité est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. lisibilité correspond à l'un de ces éléments de décision.

Exemple de code

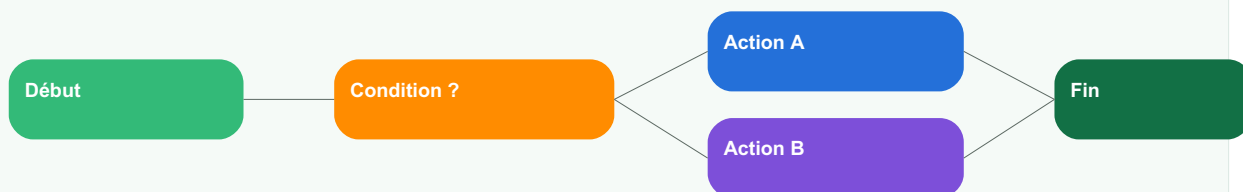
Code — Optimisation raisonnée

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à mesure. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à profiling. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à compromis. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à lisibilité. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à mesure. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à profiling. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

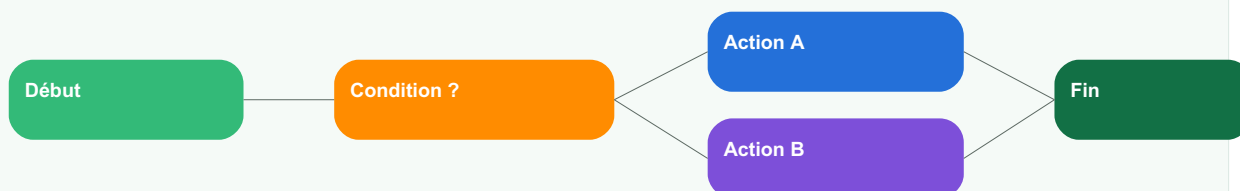
Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Deux pointeurs

Ce chapitre traite de deux pointeurs pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Décision algorithmique



Objectifs pédagogiques

- Comprendre le rôle de deux pointeurs dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
gauche	Comment utiliser gauche pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
droite	Comment utiliser droite pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
fenêtre	Comment utiliser fenêtre pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
condition	Comment utiliser condition pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

gauche

gauche est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. gauche correspond à l'un de ces éléments de décision.

Tableau et indices**droite**

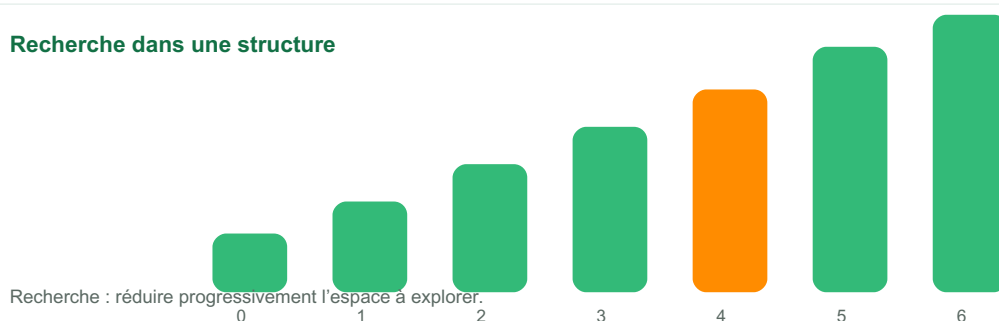
droite est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. droite correspond à l'un de ces éléments de décision.

fenêtre

fenêtre est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. fenêtre correspond à l'un de ces éléments de décision.

Recherche dans une structure**condition**

condition est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. condition correspond à l'un de ces éléments de décision.

Exemple de code

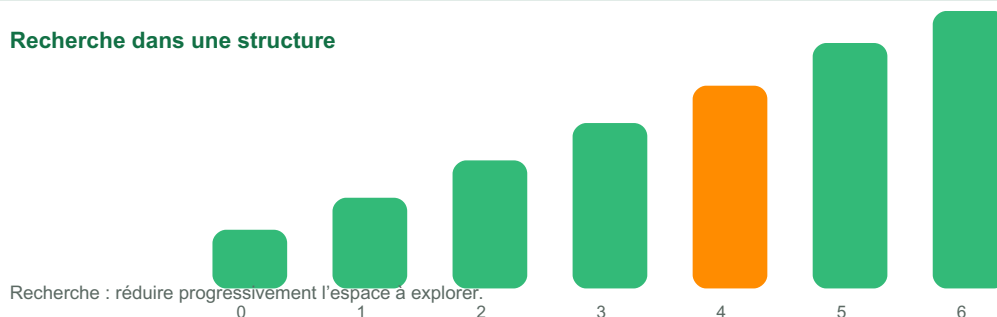
Code — Deux pointeurs

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Recherche dans une structure

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à gauche. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à droite. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à fenêtre. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à condition. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à gauche. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à droite. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Fenetre glissante

Ce chapitre traite de fenetre glissante pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Tableau et indices



Objectifs pédagogiques

- Comprendre le rôle de fenetre glissante dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
segment	Comment utiliser segment pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
somme	Comment utiliser somme pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
maximum	Comment utiliser maximum pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
mise à jour	Comment utiliser mise à jour pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

segment

segment est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. segment correspond à l'un de ces éléments de décision.

Courbes de complexité

Complexité : comparer le coût quand la taille des données augmente.

somme

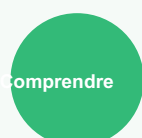
somme est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. somme correspond à l'un de ces éléments de décision.

maximum

maximum est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. maximum correspond à l'un de ces éléments de décision.

Méthode de résolution

Décomposer

Tester

Améliorer

mise à jour

mise à jour est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. mise à jour correspond à l'un de ces éléments de décision.

Exemple de code

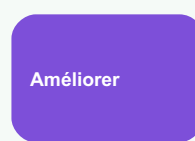
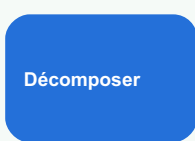
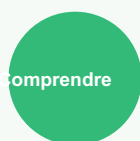
Code — Fenêtre glissante

```
def resoudre(donnees):
    resultat = []
    for element in donnees:
        if element is not None:
            resultat.append(element)
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Méthode de résolution

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à segment. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à somme. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à maximum. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à mise à jour. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à segment. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à somme. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

RéCURSIVITE

Ce chapitre traite de récursivité pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Objectifs pédagogiques

- Comprendre le rôle de récursivité dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
cas de base	Comment utiliser cas de base pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
appel	Comment utiliser appel pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
pile	Comment utiliser pile pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
retour	Comment utiliser retour pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

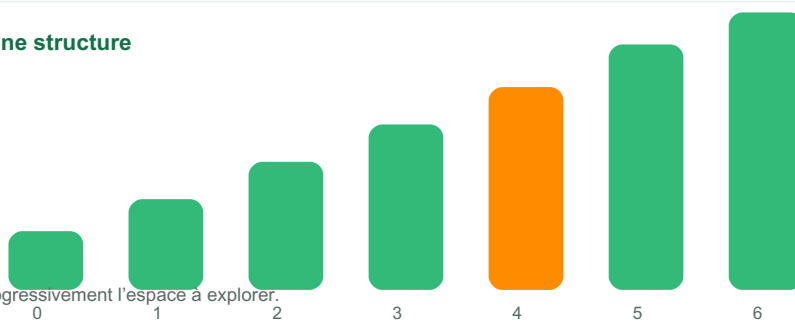
cas de base

cas de base est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. cas de base correspond à l'un de ces éléments de décision.

Recherche dans une structure

Recherche : réduire progressivement l'espace à explorer.

**appel**

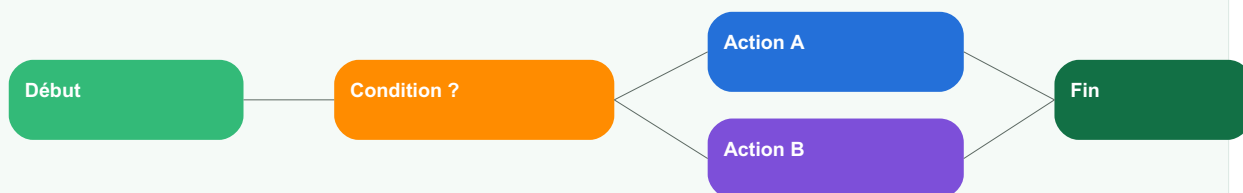
appel est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. appel correspond à l'un de ces éléments de décision.

pile

pile est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. pile correspond à l'un de ces éléments de décision.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

retour

retour est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. retour correspond à l'un de ces éléments de décision.

Exemple de code

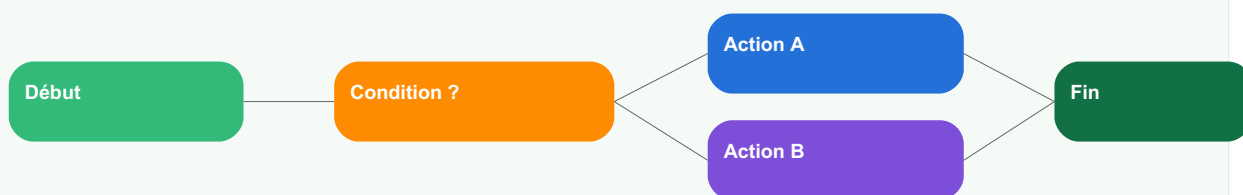
Code — Récursivité

```
def factorielle(n):  
    if n <= 1:  
        return 1  
    return n * factorielle(n - 1)
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à cas de base. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à appel. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à pile. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à retour. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à cas de base. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à appel. Donnez le pseudo-code, le code Python, trois tests et la complexité.

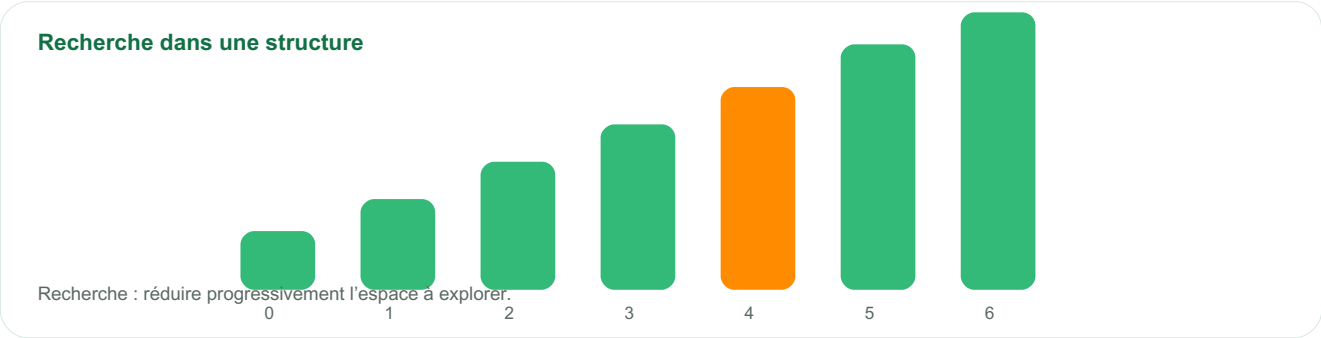
Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Programmation dynamique

Ce chapitre traite de programmation dynamique pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.



Objectifs pédagogiques

- Comprendre le rôle de programmation dynamique dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

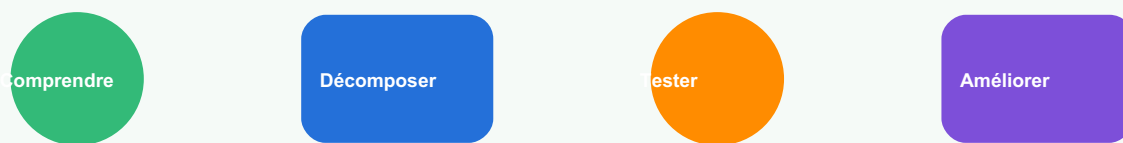
Notion	Question logique	Résultat attendu
sous-problème	Comment utiliser sous-problème pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
mémoïsation	Comment utiliser mémoïsation pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
table	Comment utiliser table pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
optimalité	Comment utiliser optimalité pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

sous-problème

sous-problème est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. sous-problème correspond à l'un de ces éléments de décision.

Méthode de résolution**mémoïsation**

mémoïsation est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. mémoïsation correspond à l'un de ces éléments de décision.

table

table est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. table correspond à l'un de ces éléments de décision.

Tableau et indices

12	5	21	9	1	17	30	3
0	1	2	3	4	5	6	7

optimalité

optimalité est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. optimalité correspond à l'un de ces éléments de décision.

Exemple de code

Code — Programmation dynamique

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Tableau et indices

12	5	21	9	1	17	30	3
0	1	2	3	4	5	6	7

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à sous-problème. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à mémorisation. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à table. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à optimalité. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à sous-problème. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à mémorisation. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

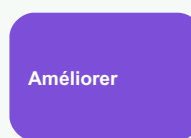
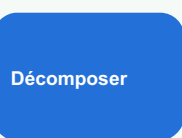
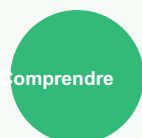
Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Graphes introductifs

Ce chapitre traite de graphes introductifs pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Méthode de résolution



Objectifs pédagogiques

- Comprendre le rôle de graphes introductifs dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
sommet	Comment utiliser sommet pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
arête	Comment utiliser arête pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
parcours	Comment utiliser parcours pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
chemin	Comment utiliser chemin pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

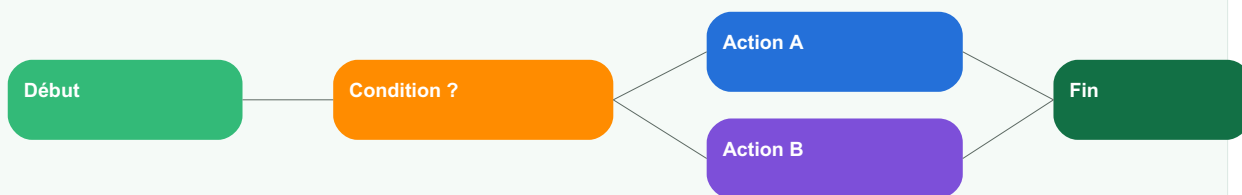
Explication approfondie

sommet

sommet est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. sommet correspond à l'un de ces éléments de décision.

Décision algorithmique



Organigramme : visualiser décision, branches et sortie.

arête

arête est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. arête correspond à l'un de ces éléments de décision.

parcours

parcours est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. parcours correspond à l'un de ces éléments de décision.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

chemin

chemin est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. chemin correspond à l'un de ces éléments de décision.

Exemple de code

Code — Graphes introductifs

```
from collections import deque

def parcours_largeur(graphe, depart):
    visites = set([depart])
    file = deque([depart])
    ordre = []
    while file:
        sommet = file.popleft()
        ordre.append(sommet)
        for voisin in graphe.get(sommet, []):
            if voisin not in visites:
                visites.add(voisin)
                file.append(voisin)
    return ordre
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.

- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à sommet. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à arête. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à parcours. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à chemin. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à sommet. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à arête. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

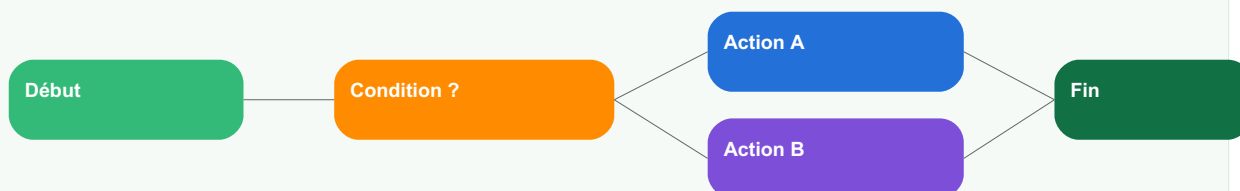
Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Moteur de recherche simple

Ce chapitre traite de moteur de recherche simple pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Décision algorithmique



Objectifs pédagogiques

- Comprendre le rôle de moteur de recherche simple dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
texte	Comment utiliser texte pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
index	Comment utiliser index pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
score	Comment utiliser score pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
résultat	Comment utiliser résultat pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

texte

texte est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. texte correspond à l'un de ces éléments de décision.

Tableau et indices**index**

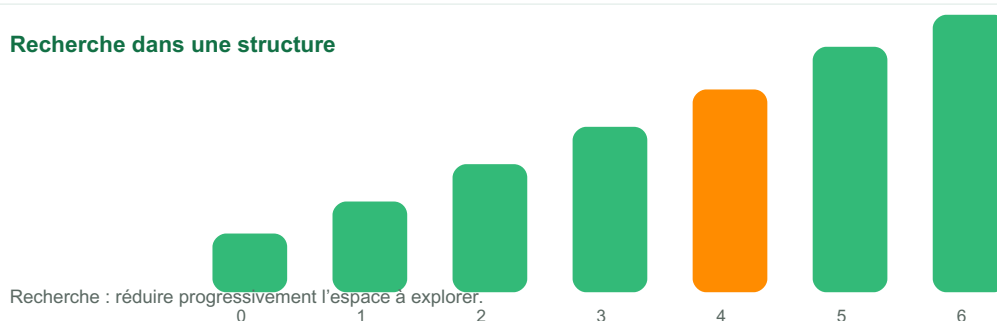
index est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. index correspond à l'un de ces éléments de décision.

score

score est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. score correspond à l'un de ces éléments de décision.

Recherche dans une structure**résultat**

résultat est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. résultat correspond à l'un de ces éléments de décision.

Exemple de code

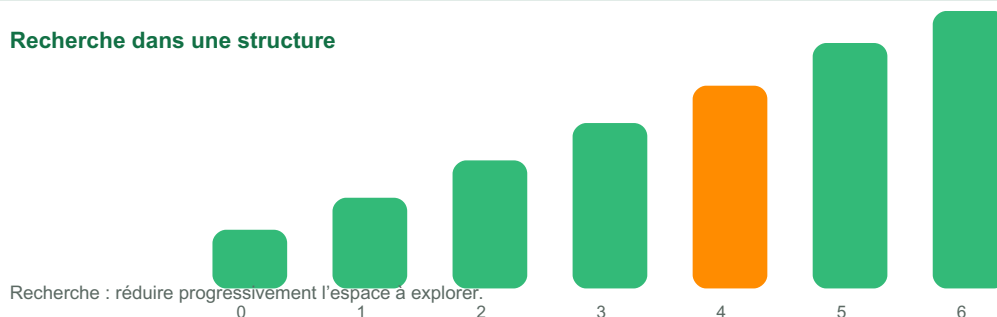
Code — Moteur de recherche simple

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Recherche dans une structure

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à texte. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à index. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à score. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à résultat. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à texte. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à index. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Analyseur de transactions

Ce chapitre traite de analyseur de transactions pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Tableau et indices



Objectifs pédagogiques

- Comprendre le rôle de analyseur de transactions dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
liste	Comment utiliser liste pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
filtre	Comment utiliser filtre pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
somme	Comment utiliser somme pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
alerte	Comment utiliser alerte pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

liste

liste est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. liste correspond à l'un de ces éléments de décision.

Courbes de complexité

Complexité : comparer le coût quand la taille des données augmente.

filtre

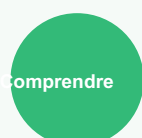
filtre est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. filtre correspond à l'un de ces éléments de décision.

somme

somme est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. somme correspond à l'un de ces éléments de décision.

Méthode de résolution

Décomposer

Tester

Améliorer

alerte

alerte est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. alerte correspond à l'un de ces éléments de décision.

Exemple de code

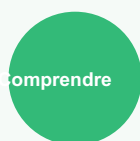
Code — Analyseur de transactions

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Méthode de résolution

Décomposer

Tester

Améliorer

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à liste. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à filtre. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à somme. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à alerte. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à liste. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à filtre. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Planificateur de tâches

Ce chapitre traite de planificateur de tâches pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Objectifs pédagogiques

- Comprendre le rôle de planificateur de tâches dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
priorité	Comment utiliser priorité pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
file	Comment utiliser file pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
temps	Comment utiliser temps pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
ordre	Comment utiliser ordre pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

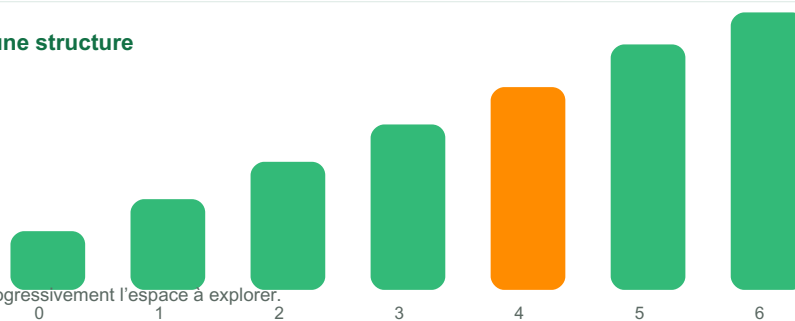
priorité

priorité est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. priorité correspond à l'un de ces éléments de décision.

Recherche dans une structure

Recherche : réduire progressivement l'espace à explorer.

**file**

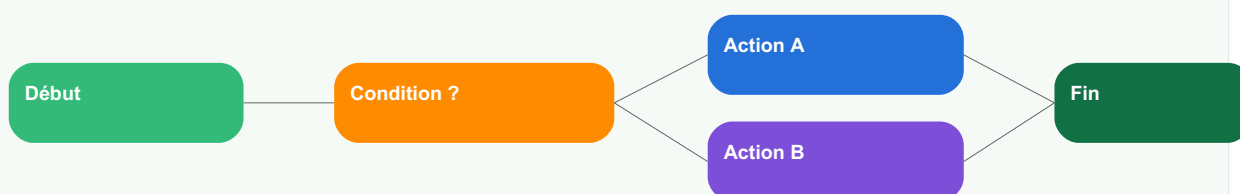
file est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. file correspond à l'un de ces éléments de décision.

temps

temps est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. temps correspond à l'un de ces éléments de décision.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

ordre

ordre est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. ordre correspond à l'un de ces éléments de décision.

Exemple de code

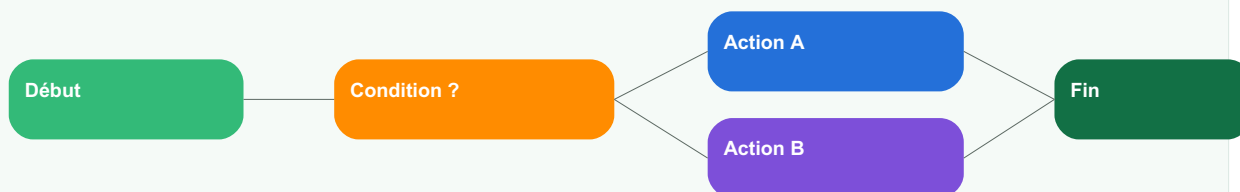
Code — Planificateur de tâches

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à priorité. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à file. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à temps. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à ordre. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à priorité. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à file. Donnez le pseudo-code, le code Python, trois tests et la complexité.

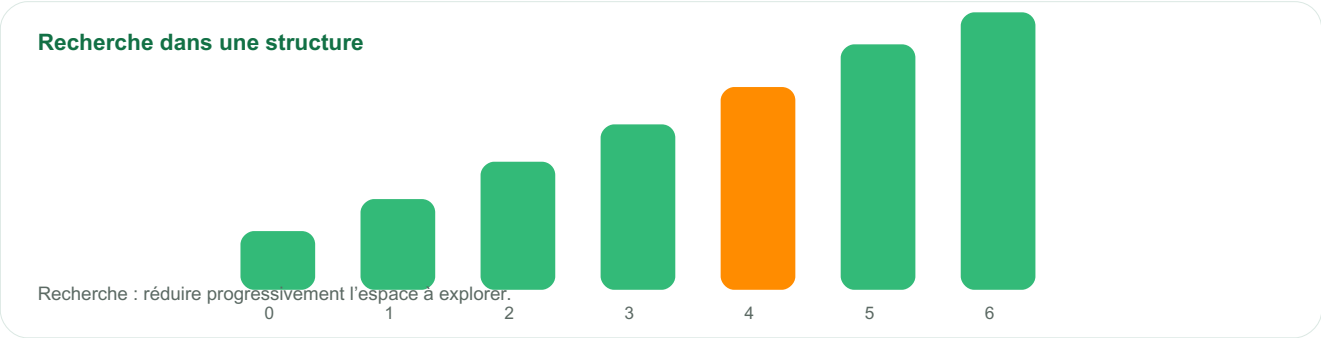
Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Mini système de recommandation

Ce chapitre traite de mini système de recommandation pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.



Objectifs pédagogiques

- Comprendre le rôle de mini système de recommandation dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

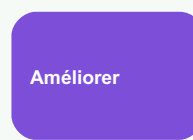
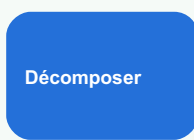
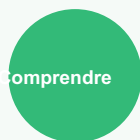
Notion	Question logique	Résultat attendu
similarité	Comment utiliser similarité pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
score	Comment utiliser score pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
tri	Comment utiliser tri pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
top	Comment utiliser top pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

similarité

similarité est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. similarité correspond à l'un de ces éléments de décision.

Méthode de résolution**score**

score est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. score correspond à l'un de ces éléments de décision.

tri

tri est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. tri correspond à l'un de ces éléments de décision.

Tableau et indices**top**

top est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. top correspond à l'un de ces éléments de décision.

Exemple de code

Code — Mini système de recommandation

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Tableau et indices

12	5	21	9	1	17	30	3
0	1	2	3	4	5	6	7

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à similarité. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à score. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à tri. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à top. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à similarité. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à score. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

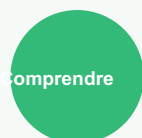
Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Jeu de logique

Ce chapitre traite de jeu de logique pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Méthode de résolution



Décomposer

Tester

Améliorer

Objectifs pédagogiques

- Comprendre le rôle de jeu de logique dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

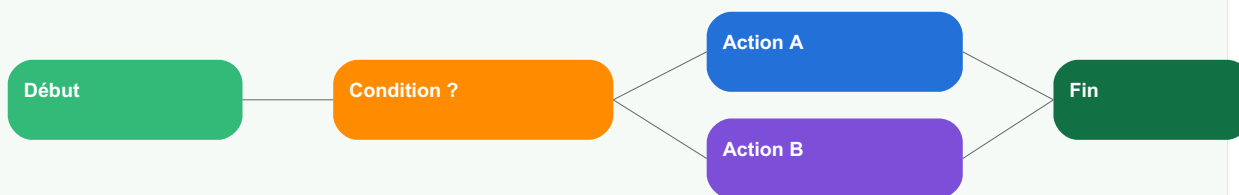
Notion	Question logique	Résultat attendu
état	Comment utiliser état pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
règle	Comment utiliser règle pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
boucle	Comment utiliser boucle pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
victoire	Comment utiliser victoire pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

état

état est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. état correspond à l'un de ces éléments de décision.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

règle

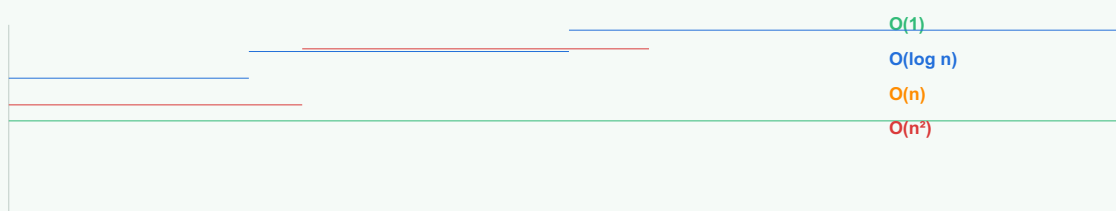
règle est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. règle correspond à l'un de ces éléments de décision.

boucle

boucle est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. boucle correspond à l'un de ces éléments de décision.

Courbes de complexité

Complexité : comparer le coût quand la taille des données augmente.

victoire

victoire est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. victoire correspond à l'un de ces éléments de décision.

Exemple de code

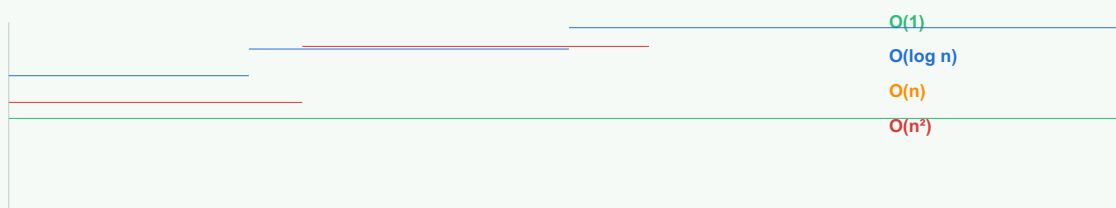
Code — Jeu de logique

```
def resoudre(donnees):
    resultat = []
    for element in donnees:
        if element is not None:
            resultat.append(element)
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Courbes de complexité

Complexité : comparer le coût quand la taille des données augmente.

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à état. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à règle. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à boucle. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à victoire. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à état. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à règle. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

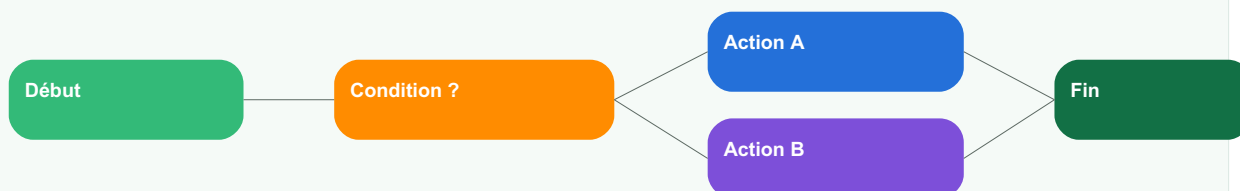
Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Banque d'exercices

Ce chapitre traite de banque d'exercices pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Décision algorithmique



Organigramme : visualiser décision, branches et sortie.

Objectifs pédagogiques

- Comprendre le rôle de banque d'exercices dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
conditions	Comment utiliser conditions pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
boucles	Comment utiliser boucles pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
tableaux	Comment utiliser tableaux pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
complexité	Comment utiliser complexité pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

conditions

conditions est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. conditions correspond à l'un de ces éléments de décision.

Tableau et indices**boucles**

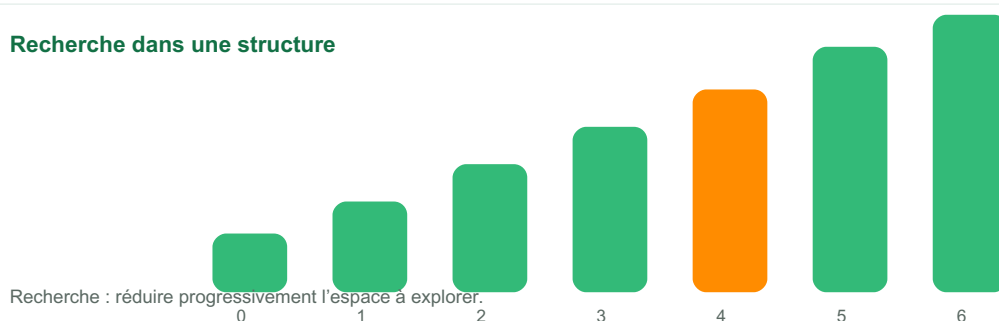
boucles est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. boucles correspond à l'un de ces éléments de décision.

tableaux

tableaux est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. tableaux correspond à l'un de ces éléments de décision.

Recherche dans une structure**complexité**

complexité est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. complexité correspond à l'un de ces éléments de décision.

Exemple de code

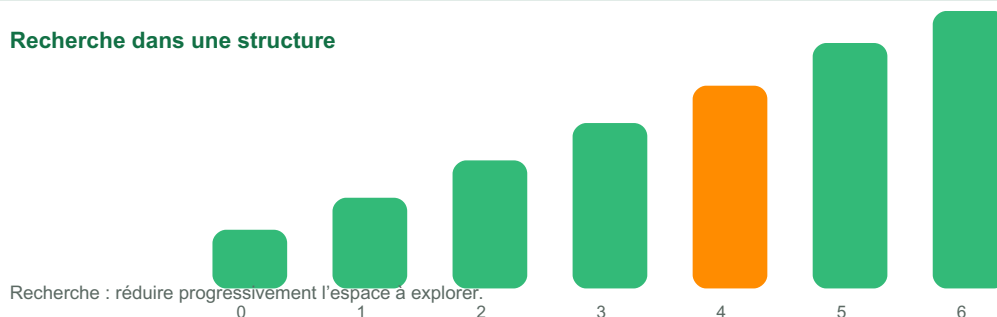
Code — Banque d'exercices

```
def resoudre(donnees):
    resultat = []
    for element in donnees:
        if element is not None:
            resultat.append(element)
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Recherche dans une structure

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à conditions. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à boucles. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à tableaux. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à complexité. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à conditions. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à boucles. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Examens blancs

Ce chapitre traite de examens blancs pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Tableau et indices



Objectifs pédagogiques

- Comprendre le rôle de examens blancs dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
problèmes	Comment utiliser problèmes pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
preuve	Comment utiliser preuve pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
code	Comment utiliser code pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
analyse	Comment utiliser analyse pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

problèmes

problèmes est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. problèmes correspond à l'un de ces éléments de décision.

Courbes de complexité

Complexité : comparer le coût quand la taille des données augmente.

preuve

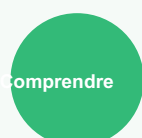
preuve est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. preuve correspond à l'un de ces éléments de décision.

code

code est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. code correspond à l'un de ces éléments de décision.

Méthode de résolution

Décomposer

Tester

Améliorer

analyse

analyse est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. analyse correspond à l'un de ces éléments de décision.

Exemple de code

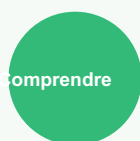
Code — Examens blancs

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Méthode de résolution

Décomposer

Tester

Améliorer

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à problèmes. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à preuve. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à code. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à analyse. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à problèmes. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à preuve. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Glossaire et feuille de route

Ce chapitre traite de glossaire et feuille de route pour renforcer la logique de programmation. L'objectif est de comprendre le raisonnement, pas seulement de mémoriser une syntaxe. Chaque notion doit être reliée à une entrée, un traitement, une sortie, des tests et une estimation du coût.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Objectifs pédagogiques

- Comprendre le rôle de glossaire et feuille de route dans la résolution de problèmes.
- Passer d'un énoncé en langage naturel à un pseudo-code clair.
- Écrire une solution Python simple, testable et lisible.
- Identifier les cas limites et les erreurs possibles.
- Comparer la solution selon le temps, la mémoire et la clarté.

Notion	Question logique	Résultat attendu
termes	Comment utiliser termes pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
méthode	Comment utiliser méthode pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
progression	Comment utiliser progression pour résoudre le problème ?	Pseudo-code, code, tests et complexité.
pratique	Comment utiliser pratique pour résoudre le problème ?	Pseudo-code, code, tests et complexité.

Explication approfondie

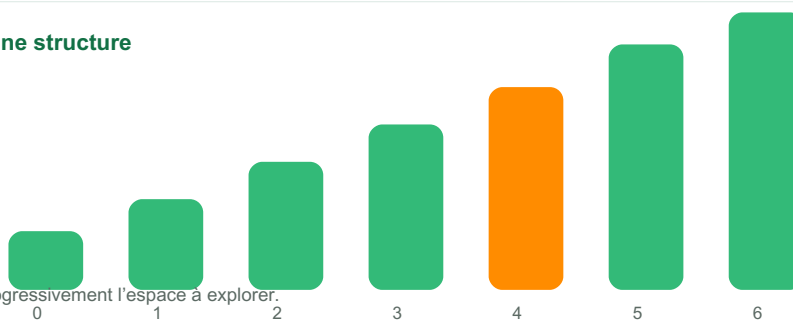
termes

termes est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. termes correspond à l'un de ces éléments de décision.

Recherche dans une structure

Recherche : réduire progressivement l'espace à explorer.

**méthode**

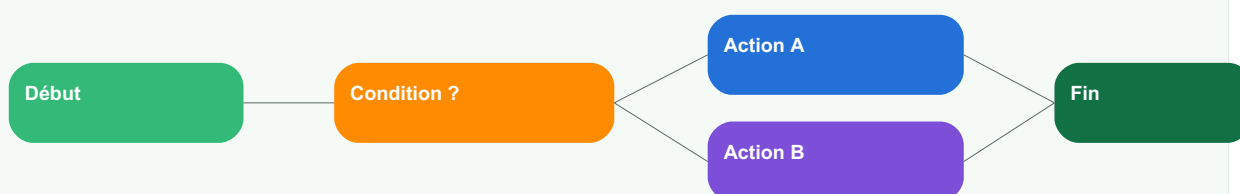
méthode est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. méthode correspond à l'un de ces éléments de décision.

progression

progression est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. progression correspond à l'un de ces éléments de décision.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

pratique

pratique est une brique de raisonnement. Un bon développeur ne commence pas par taper du code : il identifie ce qui est connu, ce qui est demandé, les contraintes, les cas particuliers et la méthode la plus simple pour avancer vers la réponse.

Analogie : résoudre un problème algorithmique ressemble à organiser un trajet. Il faut connaître le point de départ, la destination, les obstacles, les étapes et le critère d'efficacité. pratique correspond à l'un de ces éléments de décision.

Exemple de code

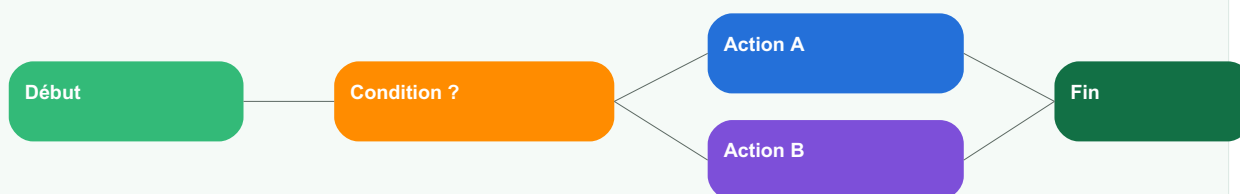
Code — Glossaire et feuille de route

```
def resoudre(donnees):  
    resultat = []  
    for element in donnees:  
        if element is not None:  
            resultat.append(element)  
    return resultat
```

Le code doit rester lisible. Une solution correcte mais incompréhensible est difficile à maintenir. Une solution professionnelle indique clairement les entrées, les sorties et la logique utilisée.

Méthode de résolution

- Lire l'énoncé et reformuler le problème.
- Identifier les entrées, sorties et contraintes.
- Écrire un exemple à la main.
- Proposer une première solution simple.
- Tester les cas limites.
- Améliorer seulement si le coût devient problématique.

Décision algorithmique

Organigramme : visualiser décision, branches et sortie.

Bonnes pratiques

- Préférer une solution claire avant une solution sophistiquée.
- Nommer les variables selon leur rôle logique.
- Éviter les boucles infinies en vérifiant la condition d'arrêt.
- Tester les tableaux vides, valeurs répétées et valeurs absentes.
- Expliquer la complexité avec des mots simples.
- Découper un grand problème en petites fonctions.

Pièges à éviter

- Coder avant de comprendre l'énoncé.
- Ignorer les cas limites.
- Confondre indice et valeur.
- Modifier une liste pendant un parcours sans raison claire.
- Optimiser prématurément au détriment de la lisibilité.

Exercices corrigés

Exercice 1 : résolvez un problème lié à termes. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 1 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 2 : résolvez un problème lié à méthode. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 2 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 3 : résolvez un problème lié à progression. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 3 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 4 : résolvez un problème lié à pratique. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 4 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 5 : résolvez un problème lié à termes. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 5 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Exercice 6 : résolvez un problème lié à méthode. Donnez le pseudo-code, le code Python, trois tests et la complexité.

Corrigé 6 : une bonne réponse reformule le problème, choisit une structure simple, vérifie les cas limites et annonce un coût approximatif. Le code doit être court et les noms explicites.

Quiz de validation

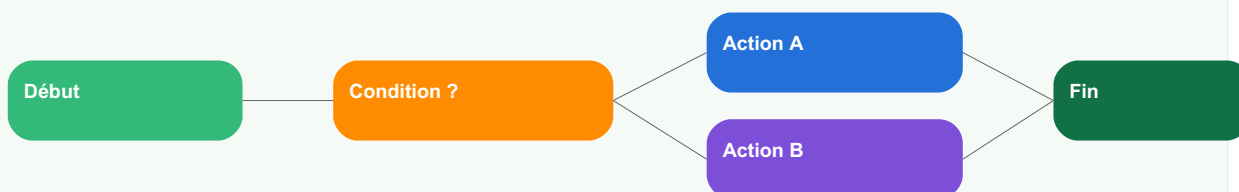
- Quelle est l'entrée du problème ?
- Quelle condition arrête l'algorithme ?
- Quel cas limite peut casser la solution ?
- Quelle est la complexité approximative ?
- Comment rendre la solution plus lisible ?

Banque de 500 exercices d'algorithmique

Cette banque d'exercices entraîne la logique : conditions, boucles, tableaux, chaînes, recherche, tri, récursivité, complexité, graphes et résolution de problèmes.

Conditions

Décision algorithmique



Organigramme : visualiser décision, branches et sortie.

Exercice 1 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 2 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 3 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 4 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 5 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 6 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 7 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 8 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 9 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 10 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 11 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 12 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 13 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 14 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 15 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 39 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 40 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 41 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 42 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 43 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 44 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 45 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 46 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 47 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 48 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 49 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 50 : résolvez un problème lié à Conditions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Boucles

Tableau et indices



Exercice 51 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 52 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 53 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 54 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 55 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 56 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 57 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 58 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 82 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 83 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 84 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 85 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 86 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 87 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 88 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 89 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 90 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 91 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 92 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 93 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 94 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 95 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 96 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 97 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

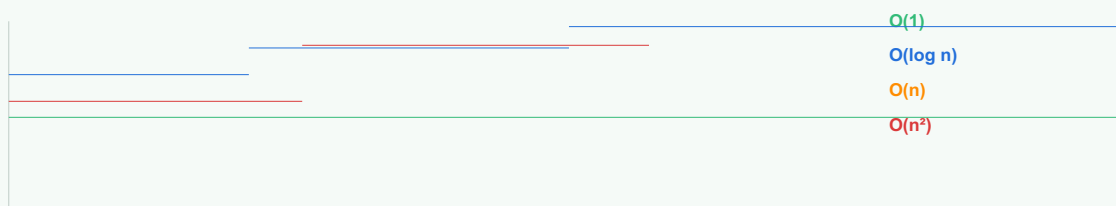
Exercice 98 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 99 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 100 : résolvez un problème lié à Boucles. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Tableaux

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Exercice 124 : résolvez un problème lié à Tableaux. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

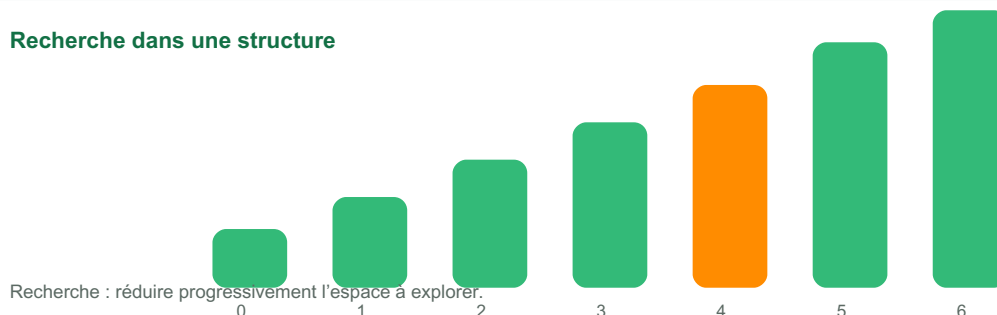
Exercice 148 : résolvez un problème lié à Tableaux. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 149 : résolvez un problème lié à Tableaux. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 150 : résolvez un problème lié à Tableaux. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Chaînes

Recherche dans une structure



Exercice 151 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 152 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 153 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 154 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 155 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 156 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 157 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 158 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 159 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 160 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 161 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 162 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 163 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 164 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 165 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 166 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 167 : résolvez un problème lié à Chaînes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
 Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 234 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 235 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 236 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 237 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 238 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 239 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 240 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 241 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 242 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 243 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 244 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 245 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 246 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 247 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

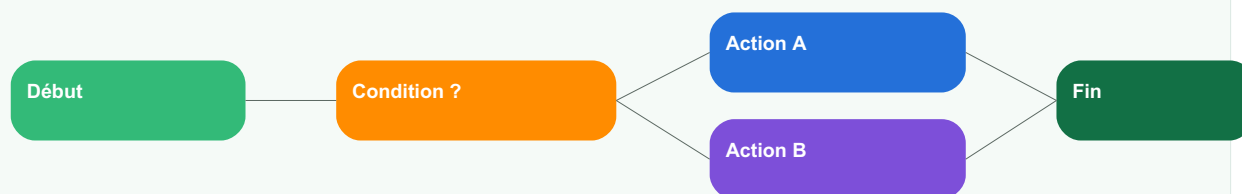
Exercice 248 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 249 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 250 : résolvez un problème lié à Fonctions. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Recherche

Décision algorithmique



Organigramme : visualiser décision, branches et sortie.

Exercice 251 : résolvez un problème lié à Recherche. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 252 : résolvez un problème lié à Recherche. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 276 : résolvez un problème lié à Recherche. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 300 : résolvez un problème lié à Recherche. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Algorithmique et logique — Charmant Nyungu

www.charmantnyunqu.com • consultant@charmantnyunqu.com • +243 835 137 837



Exercice 319 : résolvez un problème lié à Tri. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 343 : résolvez un problème lié à Tri. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 344 : résolvez un problème lié à Tri. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 345 : résolvez un problème lié à Tri. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 346 : résolvez un problème lié à Tri. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 347 : résolvez un problème lié à Tri. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 348 : résolvez un problème lié à Tri. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 349 : résolvez un problème lié à Tri. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 350 : résolvez un problème lié à Tri. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Complexité

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Exercice 351 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 352 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 353 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 354 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 355 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 356 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 357 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 358 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 359 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 360 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 361 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité. Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 385 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 386 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 387 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 388 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 389 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 390 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 391 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 392 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 393 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 394 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 395 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 396 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 397 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

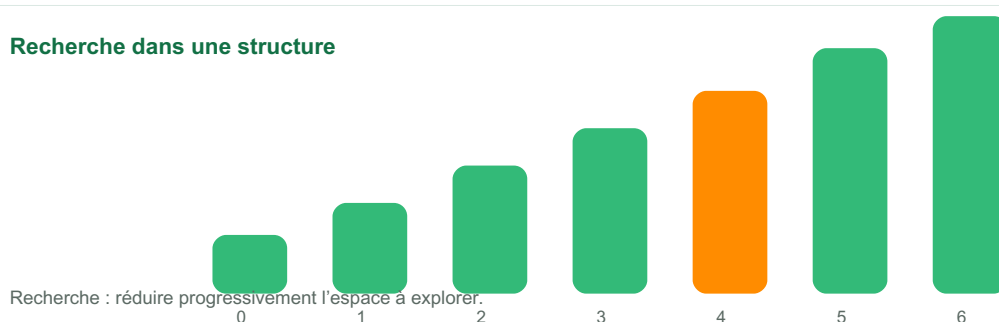
Exercice 398 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 399 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 400 : résolvez un problème lié à Complexité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Récursivité

Recherche dans une structure



Exercice 401 : résolvez un problème lié à Récursivité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 402 : résolvez un problème lié à Récursivité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 403 : résolvez un problème lié à Récursivité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 404 : résolvez un problème lié à Récursivité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 428 : résolvez un problème lié à Récursivité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 450 : résolvez un problème lié à Récursivité. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Graphes

Comprendre

Comprendre

Décomposer

ester

Améliorer

Exercice 470 : résolvez un problème lié à Graphes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 494 : résolvez un problème lié à Graphes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 495 : résolvez un problème lié à Graphes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 496 : résolvez un problème lié à Graphes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 497 : résolvez un problème lié à Graphes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 498 : résolvez un problème lié à Graphes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 499 : résolvez un problème lié à Graphes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Exercice 500 : résolvez un problème lié à Graphes. Fournissez l'analyse, le pseudo-code, le code, trois tests et la complexité.
Corrigé attendu : raisonnement clair, solution correcte, cas limites et amélioration possible.

Examens blancs

Examen blanc 1

- Partie A : lecture et compréhension d'algorithmes.
- Partie B : conditions, boucles et tableaux.
- Partie C : recherche, tri et complexité.
- Partie D : problème complet avec justification.
- Partie E : soutenance orale de la solution.

Examen blanc 2

- Partie A : lecture et compréhension d'algorithmes.
- Partie B : conditions, boucles et tableaux.
- Partie C : recherche, tri et complexité.
- Partie D : problème complet avec justification.
- Partie E : soutenance orale de la solution.

Examen blanc 3

- Partie A : lecture et compréhension d'algorithmes.
- Partie B : conditions, boucles et tableaux.
- Partie C : recherche, tri et complexité.
- Partie D : problème complet avec justification.
- Partie E : soutenance orale de la solution.

Examen blanc 4

- Partie A : lecture et compréhension d'algorithmes.
- Partie B : conditions, boucles et tableaux.
- Partie C : recherche, tri et complexité.
- Partie D : problème complet avec justification.
- Partie E : soutenance orale de la solution.

Examen blanc 5

- Partie A : lecture et compréhension d'algorithmes.

- Partie B : conditions, boucles et tableaux.
- Partie C : recherche, tri et complexité.
- Partie D : problème complet avec justification.
- Partie E : soutenance orale de la solution.

Examen blanc 6

- Partie A : lecture et compréhension d'algorithmes.
- Partie B : conditions, boucles et tableaux.
- Partie C : recherche, tri et complexité.
- Partie D : problème complet avec justification.
- Partie E : soutenance orale de la solution.

Glossaire algorithmique

Terme	Définition
Algorithme	Suite finie d'étapes pour résoudre un problème.
Condition	Test qui choisit une branche d'exécution.
Boucle	Répétition contrôlée d'instructions.
Tableau	Structure ordonnée accessible par indice.
Recherche	Méthode pour trouver une valeur ou une position.
Tri	Organisation des valeurs selon un ordre.
Complexité	Estimation du coût en temps ou mémoire.
Récursivité	Fonction qui s'appelle elle-même avec un cas d'arrêt.

Feuille de route pour devenir fort en algorithmique

- Maîtriser conditions, boucles, fonctions et tableaux.
- Résoudre 10 problèmes simples par semaine.
- Apprendre recherche, tri et complexité.
- Pratiquer récursivité, dictionnaires et graphes.
- Expliquer chaque solution avant de la coder.
- Comparer plusieurs approches et tester les cas limites.

Contact professionnel

Charmant Nyungu — Consultant en innovation technologique, transformation numérique, cybersécurité, intelligence artificielle, développement logiciel et stratégie digitale.

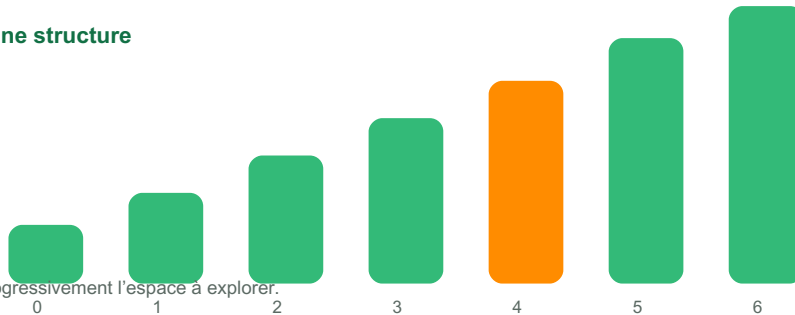
- www.charmantnyungu.com
- consultant@charmantnyungu.com
- +243 835 137 837

Fiche pratique algorithmique 158

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Recherche dans une structure

Recherche : réduire progressivement l'espace à explorer.



Étape

Question à poser

Comprendre

Que demande exactement le problème ?

Décomposer

Quelles petites étapes résolvent le problème ?

Tester

Quels cas simples, limites et invalides vérifier ?

Améliorer

Le coût est-il acceptable ?

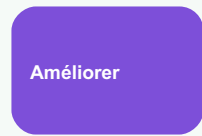
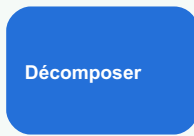
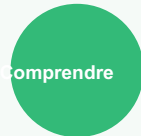
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 159

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Méthode de résolution



Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

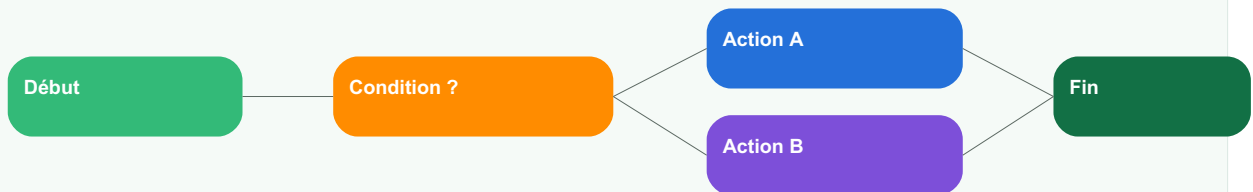
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 160

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Décision algorithmique



Organigramme : visualiser décision, branches et sortie.

Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 161

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Tableau et indices

12	5	21	9	1	17	30	3
0	1	2	3	4	5	6	7

Étape

Question à poser

Comprendre

Que demande exactement le problème ?

Décomposer

Quelles petites étapes résolvent le problème ?

Tester

Quels cas simples, limites et invalides vérifier ?

Améliorer

Le coût est-il acceptable ?

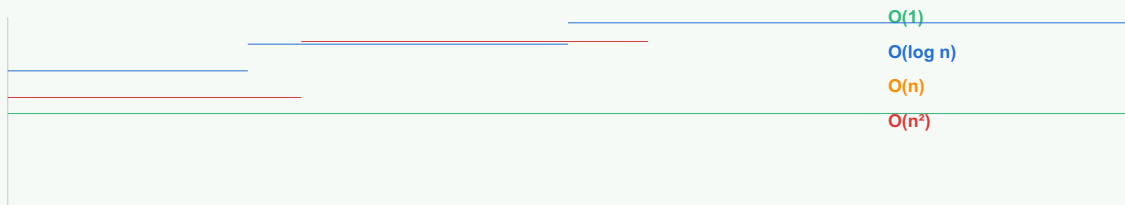
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 162

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

Gabarit de parcours linéaire

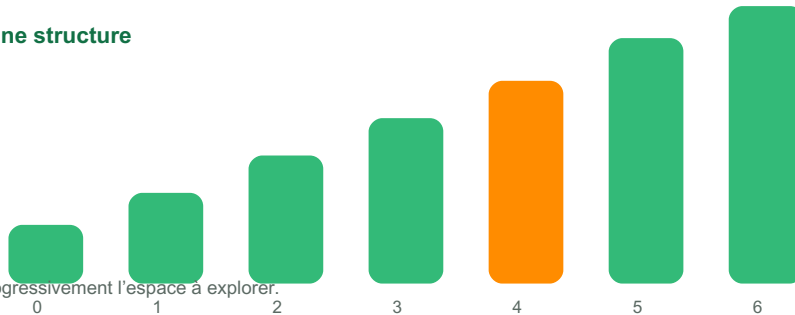
```
def compter_occurrences(valeurs, cible):
    total = 0
    for valeur in valeurs:
        if valeur == cible:
            total += 1
    return total
```

Fiche pratique algorithmique 163

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Recherche dans une structure

Recherche : réduire progressivement l'espace à explorer.



Étape

Question à poser

Comprendre

Que demande exactement le problème ?

Décomposer

Quelles petites étapes résolvent le problème ?

Tester

Quels cas simples, limites et invalides vérifier ?

Améliorer

Le coût est-il acceptable ?

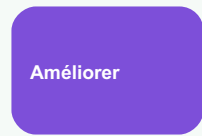
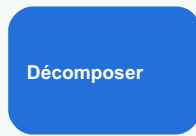
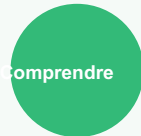
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):
    total = 0
    for valeur in valeurs:
        if valeur == cible:
            total += 1
    return total
```

Fiche pratique algorithmique 164

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Méthode de résolution



Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

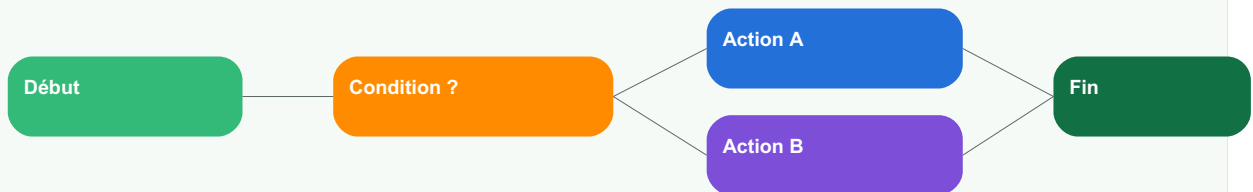
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```


Fiche pratique algorithmique 165

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Décision algorithmique



Organigramme : visualiser décision, branches et sortie.

Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 166

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Tableau et indices

12	5	21	9	1	17	30	3
0	1	2	3	4	5	6	7

Étape

Question à poser

Comprendre

Que demande exactement le problème ?

Décomposer

Quelles petites étapes résolvent le problème ?

Tester

Quels cas simples, limites et invalides vérifier ?

Améliorer

Le coût est-il acceptable ?

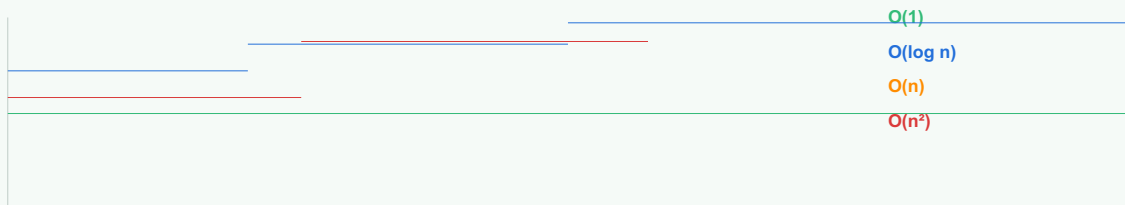
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 167

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

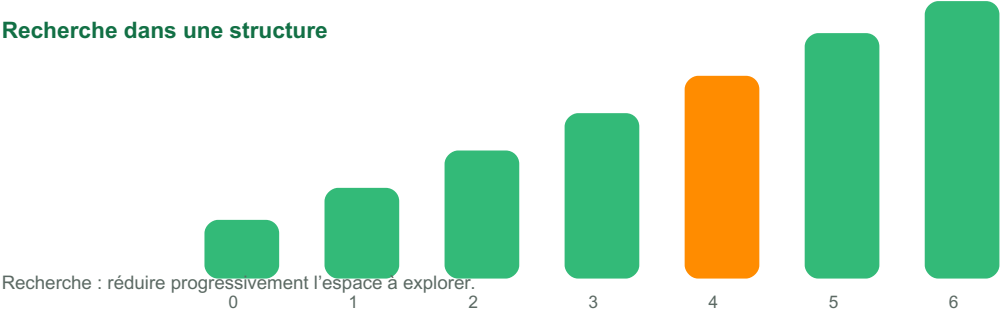
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):
    total = 0
    for valeur in valeurs:
        if valeur == cible:
            total += 1
    return total
```

Fiche pratique algorithmique 168

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Recherche dans une structure



Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

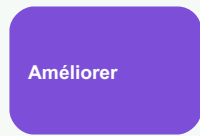
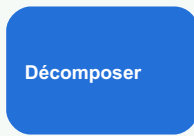
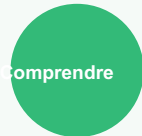
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 169

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Méthode de résolution



Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

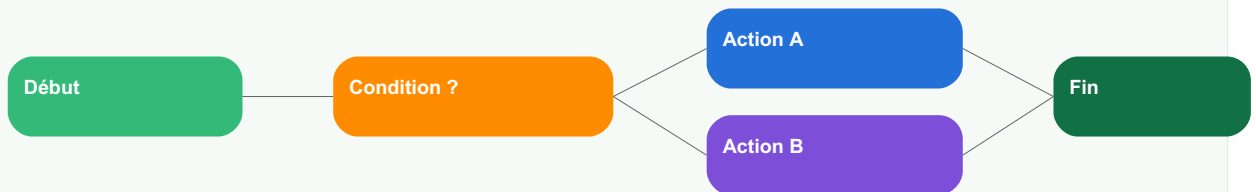
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 170

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Décision algorithmique



Organigramme : visualiser décision, branches et sortie.

Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 171

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Tableau et indices

12	5	21	9	1	17	30	3
0	1	2	3	4	5	6	7

Étape

Question à poser

Comprendre

Que demande exactement le problème ?

Décomposer

Quelles petites étapes résolvent le problème ?

Tester

Quels cas simples, limites et invalides vérifier ?

Améliorer

Le coût est-il acceptable ?

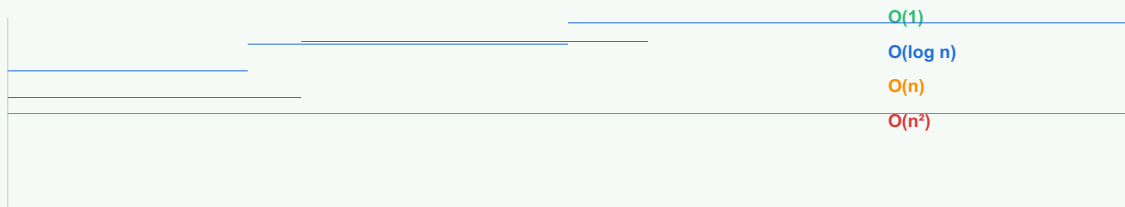
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 172

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

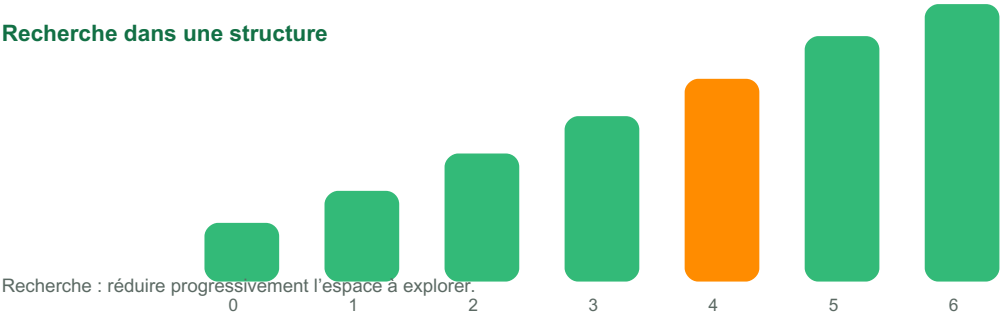
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):
    total = 0
    for valeur in valeurs:
        if valeur == cible:
            total += 1
    return total
```


Fiche pratique algorithmique 173

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Recherche dans une structure



Recherche : réduire progressivement l'espace à explorer.

Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

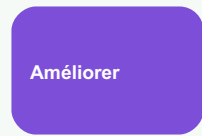
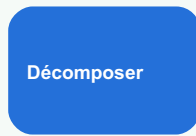
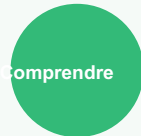
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 1 / 4

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Méthode de résolution



Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

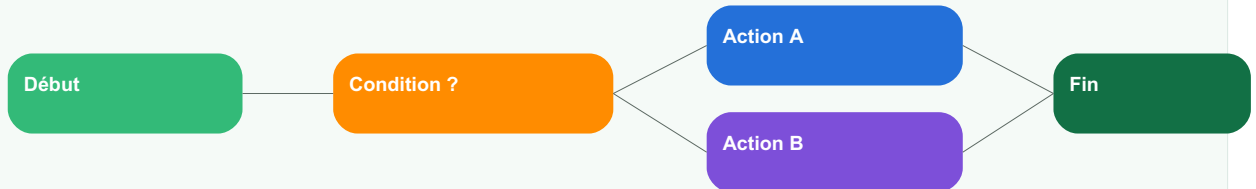
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 175

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Décision algorithmique



Organigramme : visualiser décision, branches et sortie.

Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 176

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Tableau et indices

12	5	21	9	1	17	30	3
0	1	2	3	4	5	6	7

Étape

Question à poser

Comprendre

Que demande exactement le problème ?

Décomposer

Quelles petites étapes résolvent le problème ?

Tester

Quels cas simples, limites et invalides vérifier ?

Améliorer

Le coût est-il acceptable ?

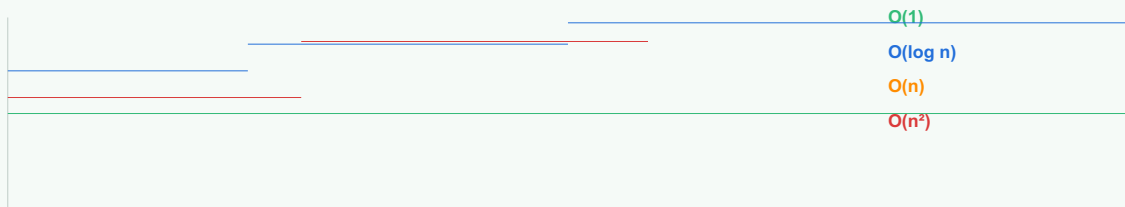
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 177

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

Gabarit de parcours linéaire

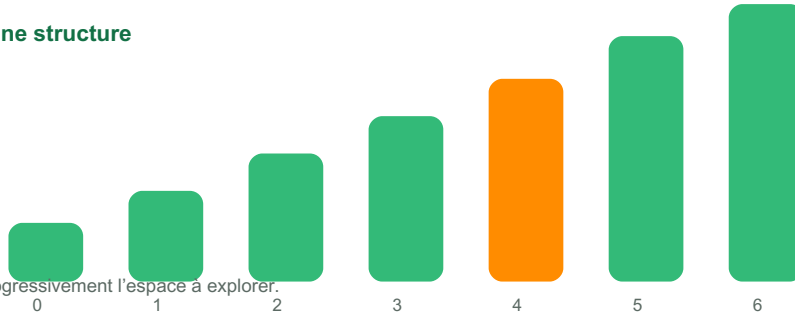
```
def compter_occurrences(valeurs, cible):
    total = 0
    for valeur in valeurs:
        if valeur == cible:
            total += 1
    return total
```

Fiche pratique algorithmique 178

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Recherche dans une structure

Recherche : réduire progressivement l'espace à explorer.



Étape

Question à poser

Comprendre

Que demande exactement le problème ?

Décomposer

Quelles petites étapes résolvent le problème ?

Tester

Quels cas simples, limites et invalides vérifier ?

Améliorer

Le coût est-il acceptable ?

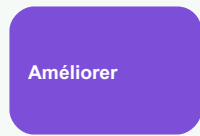
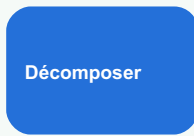
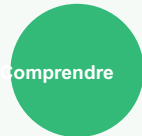
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):
    total = 0
    for valeur in valeurs:
        if valeur == cible:
            total += 1
    return total
```

Fiche pratique algorithmique 179

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Méthode de résolution



Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

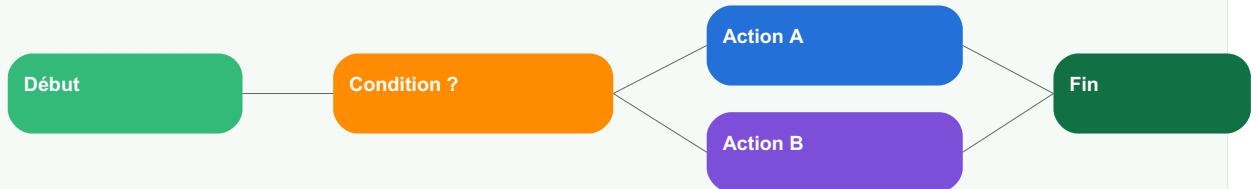
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 180

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Décision algorithmique



Organigramme : visualiser décision, branches et sortie.

Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```


Fiche pratique algorithmique 181

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Tableau et indices

12	5	21	9	1	17	30	3
0	1	2	3	4	5	6	7

Étape

Question à poser

Comprendre

Que demande exactement le problème ?

Décomposer

Quelles petites étapes résolvent le problème ?

Tester

Quels cas simples, limites et invalides vérifier ?

Améliorer

Le coût est-il acceptable ?

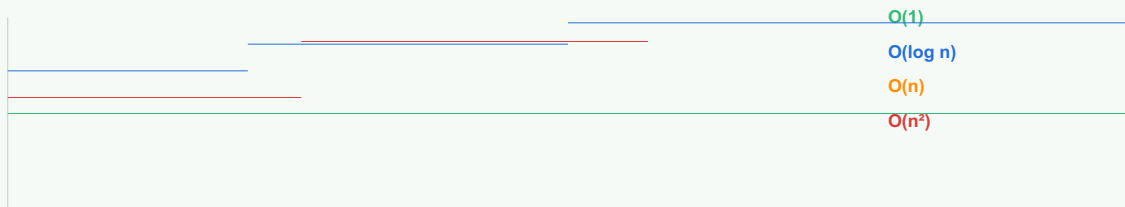
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 182

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Courbes de complexité



Complexité : comparer le coût quand la taille des données augmente.

Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

Gabarit de parcours linéaire

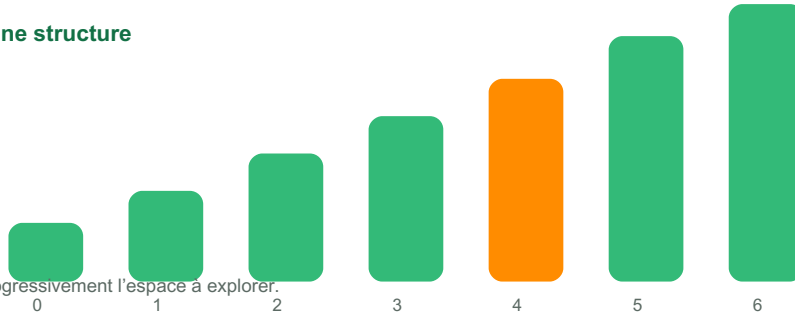
```
def compter_occurrences(valeurs, cible):
    total = 0
    for valeur in valeurs:
        if valeur == cible:
            total += 1
    return total
```

Fiche pratique algorithmique 183

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Recherche dans une structure

Recherche : réduire progressivement l'espace à explorer.



Étape

Question à poser

Comprendre

Que demande exactement le problème ?

Décomposer

Quelles petites étapes résolvent le problème ?

Tester

Quels cas simples, limites et invalides vérifier ?

Améliorer

Le coût est-il acceptable ?

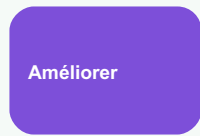
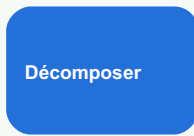
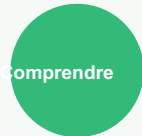
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 184

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Méthode de résolution



Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

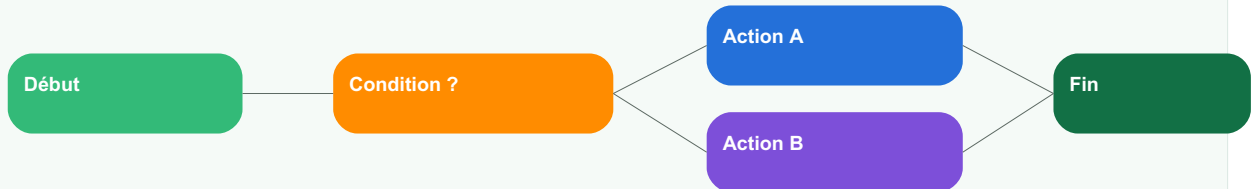
Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Fiche pratique algorithmique 185

Cette fiche sert à entraîner le raisonnement. L'apprenant doit partir d'un énoncé simple, identifier les entrées, écrire un pseudo-code, coder, tester et analyser le coût.

Décision algorithmique



Organigramme : visualiser décision, branches et sortie.

Étape	Question à poser
Comprendre	Que demande exactement le problème ?
Décomposer	Quelles petites étapes résolvent le problème ?
Tester	Quels cas simples, limites et invalides vérifier ?
Améliorer	Le coût est-il acceptable ?

Gabarit de parcours linéaire

```
def compter_occurrences(valeurs, cible):  
    total = 0  
    for valeur in valeurs:  
        if valeur == cible:  
            total += 1  
    return total
```

Organigramme conditionnel

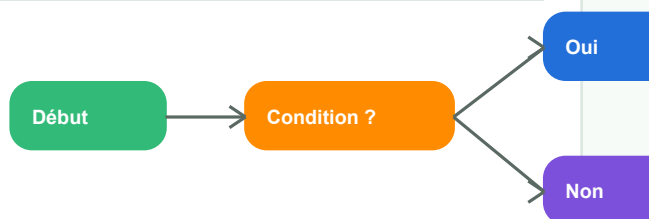
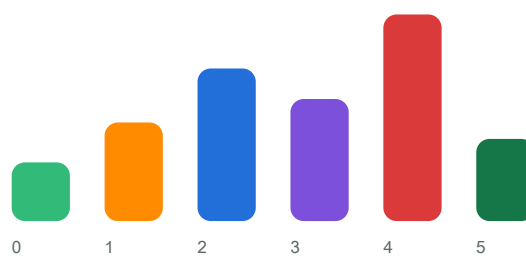
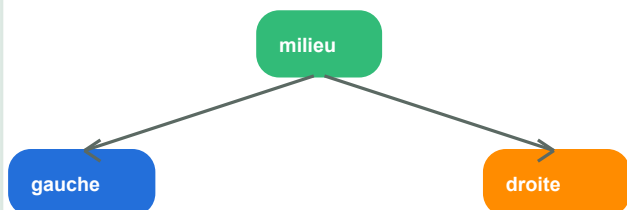


Tableau et indices

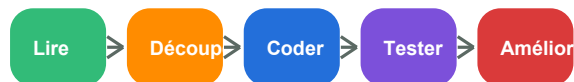


Recherche binaire



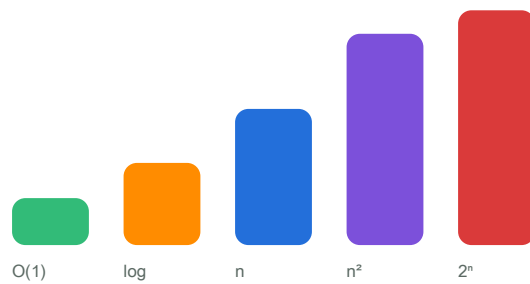
Structure hiérarchique : diviser l'espace de recherche.

Méthode de résolution



Note pédagogique : chaque diagramme est volontairement séparé dans une carte dédiée afin d'éviter tout chevauchement et de faciliter la lecture en projection ou impression.

Complexité comparative



Tri par étapes



Récursivité

appel 1

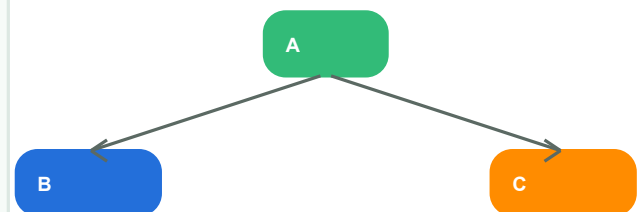
appel 2

appel 3

cas base

retour

Parcours de graphe



Structure hiérarchique : diviser l'espace de recherche.

Note pédagogique : chaque diagramme est volontairement séparé dans une carte dédiée afin d'éviter tout chevauchement et de faciliter la lecture en projection ou impression.